INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO PAULO
CAMPUS SÃO PAULO
TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

JOÃO PEDRO RODRIGUES
JOSEE HOSHI VELLANI BELLINTANI
LEONARDO GUTIERREZ MAMANI
LUIZ EDUARDO BARBOSA SILVEIRA
MURILLO ANDRADE ASSIS
YASMIN BONFIM DE MORAES

KARAOKÊ APACHE: SISTEMA INTEGRADO DE GERENCIAMENTO DE FILA, PEDIDOS MUSICAIS E REPRODUÇÃO PARA BAR DE KARAOKÊ

SÃO PAULO, SP
2026

JOÃO PEDRO RODRIGUES
JOSEE HOSHI VELLANI BELLINTANI
LEONARDO GUTIERREZ MAMANI
LUIZ EDUARDO BARBOSA SILVEIRA
MURILLO ANDRADE ASSIS
YASMIN BONFIM DE MORAES

KARAOKÊ APACHE: SISTEMA INTEGRADO DE GERENCIAMENTO DE FILA, PEDIDOS MUSICAIS E REPRODUÇÃO PARA BAR DE KARAOKÊ

Relatório Técnico apresentado como exigência parcial para avaliação na disciplina de Projeto Integrado de Extensão do Curso de Tecnologia em Análise e Desenvolvimento de Sistemas do Instituto Federal de Educação, Ciência e Tecnologia de São Paulo, Campus São Paulo.

Orientador: Prof. M^{e}. Marcelo Tavares de Santana

SÃO PAULO, SP
2026

FICHA CATALOGRÁFICA

Sistema de Bibliotecas – Instituto Federal de São Paulo (IFSP – Campus São Paulo)

R696k Rodrigues, João Pedro et al.

Karaokê Apache: Sistema Integrado de Gerenciamento de Fila, Pedidos Musicais e Reprodução para Bar de Karaokê / João Pedro Rodrigues, Josee Hoshi Vellani Bellintani, Leonardo Gutierrez Mamani, Luiz Eduardo Barbosa Silveira, Murillo Andrade Assis, Yasmin Bonfim de Moraes. – São Paulo, SP, 2026.

61 p. : il. (algumas color.) ; 30 cm.

Orientador: Prof. Me. Marcelo Tavares de Santana.

Relatório Técnico (Projeto Integrado de Extensão) – Instituto Federal de Educação, Ciência e Tecnologia de São Paulo (IFSP), Campus São Paulo, Curso de Tecnologia em Análise e Desenvolvimento de Sistemas, 2026.

1. Engenharia de Software. 2. Sistema de Karaokê. 3. Electron. 4. React. 5. Projeto de Extensão. I. Santana, Marcelo Tavares de. II. Instituto Federal de São Paulo. III. Título.

CDD 005.1

JOÃO PEDRO RODRIGUES
JOSEE HOSHI VELLANI BELLINTANI
LEONARDO GUTIERREZ MAMANI
LUIZ EDUARDO BARBOSA SILVEIRA
MURILLO ANDRADE ASSIS
YASMIN BONFIM DE MORAES

KARAOKÊ APACHE: SISTEMA INTEGRADO DE GERENCIAMENTO DE FILA, PEDIDOS MUSICAIS E REPRODUÇÃO PARA BAR DE KARAOKÊ

Relatório Técnico apresentado como exigência parcial para avaliação na disciplina de Projeto Integrado de Extensão do Curso de Tecnologia em Análise e Desenvolvimento de Sistemas do Instituto Federal de Educação, Ciência e Tecnologia de São Paulo, Campus São Paulo.

Aprovada em _____/_____/_____.

BANCA EXAMINADORA:

Prof. M^{e}. Marcelo Tavares de Santana(Orientador)
Instituto Federal de Educação, Ciência e Tecnologia de São Paulo (IFSP)

Prof. M^{e}. Johnata Souza Santicioli
Instituto Federal de Educação, Ciência e Tecnologia de São Paulo (IFSP)

Prof. M^{e}. Membro Avaliador Convidado
Instituto Federal de Educação, Ciência e Tecnologia de São Paulo (IFSP)

SÃO PAULO, SP
2026

Dedicamos este projeto aos gestores, músicos e frequentadores do estabelecimento parceiro Apache Gru Rock Bar, em Guarulhos – SP, cuja receptividade e parceria tornaram possível aplicar a engenharia de software na transformação de uma operação cultural e gastronômica real.

Dedicamos também aos pequenos empreendedores do setor de bares e entretenimento que buscam na tecnologia acessível um instrumento de organização e valorização da cultura local.

AGRADECIMENTOS

Expressamos nossos sinceros agradecimentos a todos os professores e profissionais do Instituto Federal de São Paulo pelo apoio e pela orientação ao longo de todo o percurso da graduação, sobretudo ao Prof. Marcelo Tavares de Santana, cujas orientações técnicas e acadêmicas guiaram cada etapa do desenvolvimento deste projeto.

Ao Instituto Federal de Educação, Ciência e Tecnologia de São Paulo (IFSP), Campus São Paulo (SPO), por nos proporcionar uma infraestrutura de excelência e o ambiente ideal para a aprendizagem profissional e acadêmica.

À empresa parceira Apache Gru Rock Bar, localizada no bairro Gopouva, em Guarulhos – SP, por compartilhar os desafios reais de sua operação, permitindo-nos devolver à sociedade o conhecimento técnico desenvolvido durante o curso.

Por fim, agradecemos às nossas famílias e aos nossos companheiros por todo o apoio e suporte socioemocional ao longo de toda a trajetória da graduação.

"A engenharia de software cumpre seu propósito mais nobre quando transcende a complexidade técnica do código e se converte em fluidez, organização e experiência coletiva no mundo real."

— Engenharia de Software Aplicada

RESUMO

Este relatório técnico documenta o desenvolvimento do sistema **Karaokê Apache**, projeto de extensão realizado para o **Apache Gru Rock Bar**, em Guarulhos – SP. Na Noite de Karaokê do estabelecimento, os pedidos de música eram feitos verbalmente ou em papel, e o organizador pesquisava cada vídeo manualmente no YouTube pelo projetor, o que causava perda de pedidos, repetição de músicas e falta de visibilidade sobre a ordem de apresentação. O objetivo do projeto é desenvolver o Produto Mínimo Viável (MVP) de um sistema de baixo custo para receber pedidos, organizar a fila de cantores e reproduzir os vídeos no telão. A partir do levantamento de requisitos com o parceiro, foram definidas cinco regras de negócio, entre elas a ordem de chegada (FIFO), o bloqueio de músicas repetidas e o limite de pedidos por cliente, e foi elaborada a modelagem UML do sistema. A arquitetura reúne interfaces web em React para clientes e administrador, acessadas pelo QR Code exibido no telão; uma API em Java 21 com Spring Boot e banco de dados MySQL, hospedados em nuvem, que pesquisa os vídeos pela *YouTube Data API v3*; e um reprodutor em Electron, executado no computador do bar, que exibe no telão o vídeo da música da vez ao lado da interface do sistema. As mudanças na fila são enviadas a todas as telas por WebSocket com o protocolo STOMP. A Prova de Conceito validou a sincronização em tempo real entre a interface web e o reprodutor, e o MVP encontra-se em implementação, com testes planejados para as regras de negócio, o controle de acesso e o tempo de atualização. O único custo previsto para o estabelecimento é o da hospedagem em nuvem.

Palavras-chave: Karaokê; Gerenciamento de fila; Aplicação web; Electron; Extensão universitária.

ABSTRACT

This technical report documents the development of the **Karaokê Apache** system, a university extension project carried out for the **Apache Gru Rock Bar**, in Guarulhos, São Paulo, Brazil. During the venue's Karaoke Night, song requests were made verbally or on paper, and the host searched for each video manually on YouTube through the projector, which led to lost requests, repeated songs and a lack of visibility over the singing order. The goal of the project is to develop the Minimum Viable Product (MVP) of a low-cost system to receive requests, organize the singer queue and play the videos on the main screen. Based on requirements gathered with the partner, five business rules were defined, including first-come, first-served ordering (FIFO), blocking of repeated songs and a limit of requests per customer, and the system was modeled with UML. The architecture comprises React web interfaces for customers and the host, accessed through a QR Code shown on the main screen; a Java 21 API built with Spring Boot and a MySQL database, both hosted in the cloud, which searches videos through the *YouTube Data API v3*; and an Electron player, running on the bar's computer, which shows the current song's video next to the system's own interface on the main screen. Queue changes are pushed to all screens over WebSocket using the STOMP protocol. The Proof of Concept validated real-time synchronization between the web interface and the player, and the MVP is under implementation, with tests planned for the business rules, access control and update time. The only expected cost for the venue is cloud hosting.

Keywords: Karaoke; Queue management; Web application; Electron; University extension.

LISTA DE ILUSTRAÇÕES

Figura 1 – Diagrama de Atividades UML – Fluxo de Pedido Musical e Execução na Fila 25
Figura 2 – Diagrama de Casos de Uso UML – Sistema Karaokê Apache 28
Figura 3 – Diagrama de Classes Conceitual UML (Derivado de `Conceitual.brM3`) 30
Figura 4 – Diagrama de Componentes UML 35
Figura 5 – Diagrama de Classes de Projeto UML 38
Figura 6 – *Wireframe* das Interfaces do Sistema 42
Figura 7 – Diagrama de Sequência UML – Pedido de Música pelo Cliente 44
Figura 8 – Diagrama de Sequência UML – Controle do Palco e Reprodução no Telão 45
Figura 9 – Diagrama de Estados UML – Ciclo de Vida de um Pedido Musical na Fila (RN03 e RN04) 49
Figura 10 – Diagrama de Implantação UML – Topologia de Execução do MVP . . 53

LISTA DE TABELAS

Tabela 1 – Estudo comparativo entre soluções existentes no mercado e o sistema proposto. 22
Tabela 2 – Regras de negócio do domínio do sistema Karaokê Apache. 24
Tabela 3 – Identificação e caracterização dos atores do sistema Karaokê Apache. . 27
Tabela 4 – Especificação dos casos de uso (UC) do sistema Karaokê Apache. . . . 29
Tabela 5 – Requisitos funcionais (RF) do sistema Karaokê Apache. 31
Tabela 6 – Requisitos não funcionais (RNF) do sistema Karaokê Apache. 32
Tabela 7 – Correspondência entre subsistemas e componentes lógicos. 36
Tabela 8 – Detalhamento técnico dos componentes arquiteturais e suas interfaces. 37
Tabela 9 – Dicionário de dados do modelo relacional do sistema Karaokê Apache. 39
Tabela 10 – Tecnologias, frameworks e ferramentas utilizadas no projeto. 47
Tabela 11 – Testes realizados na Prova de Conceito e testes planejados para o MVP. 54

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
Abrasel	Associação Brasileira de Bares e Restaurantes
API	*Application Programming Interface*
Caged	Cadastro Geral de Empregados e Desempregados
CEP	Código de Endereçamento Postal
CNAE	Classificação Nacional de Atividades Econômicas
CNE/CES	Conselho Nacional de Educação / Câmara de Educação Superior
CNPJ	Cadastro Nacional da Pessoa Jurídica
CORS	*Cross-Origin Resource Sharing*
CSP	*Content Security Policy*
FDD	*Feature-Driven Development*
FIFO	*First In, First Out*
FK	*Foreign Key* (chave estrangeira)
HTTP	*Hypertext Transfer Protocol*
HTTPS	*Hypertext Transfer Protocol Secure*
IFSP	Instituto Federal de Educação, Ciência e Tecnologia de São Paulo
JDBC	*Java Database Connectivity*
JVM	*Java Virtual Machine*
JWT	*JSON Web Token*
LGPD	Lei Geral de Proteção de Dados Pessoais (Lei nº 13.709/2018)
MPMEs	Micro, Pequenas e Médias Empresas
MVP	*Minimum Viable Product* (Produto Mínimo Viável)
PK	*Primary Key* (chave primária)
PoC	*Proof of Concept* (Prova de Conceito)

QR Code	*Quick Response Code*
RBAC	*Role-Based Access Control*
REST	*Representational State Transfer*
RF	Requisito Funcional
RN	Regra de Negócio
RNF	Requisito Não Funcional
Sebrae	Serviço Brasileiro de Apoio às Micro e Pequenas Empresas
STOMP	*Simple Text Oriented Messaging Protocol*
UC	Caso de Uso (*Use Case*)
UML	*Unified Modeling Language*
URL	*Uniform Resource Locator*
WSS	*WebSocket Secure*

LISTA DE SÍMBOLOS

N	Quantidade total de pedidos musicais ativos na fila de reprodução
$(1, 1)$	Cardinalidade mínima e máxima unitária no Modelo Entidade-Relacionamento
$(0, n)$	Cardinalidade opcional com múltiplos registros associados no Modelo Entidade-Relacionamento

SUMÁRIO

1	**INTRODUÇÃO**	**16**
1.1	Caracterização do Projeto	16
1.2	Objetivos	16
1.2.1	Objetivo Geral	17
1.2.2	Objetivos Específicos	17
1.3	Justificativa	17
1.4	Escopo do Projeto	18
1.5	Estrutura do Documento	19
2	**CONTEXTO DO PROJETO**	**21**
2.1	Cenário	22
2.2	Público-Alvo	22
2.3	Necessidades Identificadas	23
2.4	Regras de Negócio	24
3	**ANÁLISE DO SISTEMA**	**26**
3.1	Visão Geral do Sistema	26
3.2	Atores Envolvidos	26
3.3	Casos de Uso	27
3.4	Modelo Conceitual	29
3.5	Requisitos Funcionais	30
3.6	Requisitos Não Funcionais	31
3.7	Restrições do Sistema	32
4	**PROJETO DO SISTEMA**	**34**
4.1	Arquitetura do Sistema	34
4.2	Projeto dos Componentes	35
4.3	Modelo de Dados	38
4.4	Projeto da Interface do Usuário	40
4.5	Diagramas de Interação	42
4.6	Tecnologias Utilizadas	46
4.7	Segurança e Controle de Acesso	47
5	**IMPLEMENTAÇÃO**	**50**
5.1	Ambiente de Desenvolvimento	50
5.2	Estrutura da Aplicação	50
5.3	Implementação dos Componentes	51

5.4 Integração dos Módulos 52
5.5 Testes Realizados 53
5.6 Resultados Obtidos 54
5.7 Limitações da Implementação 55

6 CONCLUSÃO 57
6.1 Considerações Finais 57
6.2 Dificuldades Encontradas 57
6.3 Trabalhos Futuros 58

REFERÊNCIAS 59

ANEXO A – REFERÊNCIA NORMATIVA – ART. 6º DA LEI GERAL DE PROTEÇÃO DE DADOS PESSOAIS (LEI Nº 13.709/2018) 61

1 INTRODUÇÃO

Este capítulo apresenta o projeto **Karaokê Apache** de forma geral, estabelecendo sua caracterização, seus objetivos, sua justificativa, a delimitação de seu escopo atual em nível de Produto Mínimo Viável (MVP) e a organização estrutural do documento.

1.1 CARACTERIZAÇÃO DO PROJETO

O projeto **Karaokê Apache** consiste no desenvolvimento de uma solução computacional integrada, composta por aplicações web responsivas, por uma API hospedada em nuvem e por um reprodutor desktop dedicado ao telão, voltada à automação do gerenciamento de fila de cantores, recepção de pedidos musicais e reprodução audiovisual em eventos de karaokê. O sistema nasce como uma iniciativa acadêmica e extensionista vinculada à disciplina de *Projeto Integrado de Extensão* do Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas do Instituto Federal de Educação, Ciência e Tecnologia de São Paulo (IFSP – Campus São Paulo).

A demanda originou-se da observação direta da rotina operacional do estabelecimento parceiro **Apache Gru Rock Bar**, localizado no município de Guarulhos – SP. Durante a realização semanal de sua tradicional *Noite de Karaokê*, constatou-se que todo o fluxo de inscrição de participantes, escolha de faixas musicais e acionamento de vídeos no telão ocorria de forma manual, gerando gargalos de atendimento e sobrecarga para a equipe do bar.

Para responder a esse cenário, o projeto propõe uma arquitetura distribuída que descentraliza a entrada de pedidos, permitindo que os próprios clientes pesquisem e escolham vídeos musicais pelo sistema em seus smartphones, a partir da leitura do QR Code exibido no telão. A moderação da fila fica com o organizador, em uma interface web própria, e a reprodução dos vídeos fica com o aplicativo desktop **Karaokê TV Player**, construído com Electron e executado no computador do bar, que exibe no telão o vídeo da música da vez ao lado da interface do sistema. O projeto encontra-se em fase de desenvolvimento do seu **Produto Mínimo Viável (MVP)**, com a modelagem de engenharia de software concluída e a Prova de Conceito (PoC) validada.

1.2 OBJETIVOS

Os objetivos a seguir orientam o desenvolvimento do projeto e delimitam os resultados alcançados na modelagem e na construção do MVP.

1.2.1 Objetivo Geral

Projetar e desenvolver o Produto Mínimo Viável (MVP) de um sistema integrado web e desktop para o gerenciamento de fila, recepção de pedidos musicais e reprodução audiovisual automatizada nas noites de karaokê do estabelecimento Apache Gru Rock Bar, substituindo controles manuais em papel por uma solução digital organizada, acessível e de baixo custo.

1.2.2 Objetivos Específicos

- Diagnosticar o cenário operacional, o público-alvo e as necessidades do estabelecimento parceiro Apache Gru Rock Bar durante a condução da Noite de Karaokê;
- Especificar as regras de negócio (RN00 a RN04), casos de uso, requisitos funcionais (RF01 a RF12), requisitos não funcionais (RNF01 a RNF08) e restrições do sistema;
- Elaborar a modelagem completa de análise e projeto do sistema utilizando a Linguagem de Modelagem Unificada (UML) e modelagem conceitual/relacional de banco de dados;
- Implementar a Prova de Conceito (PoC) e o núcleo do MVP integrando a interface web de controle e fila em React/Vite com o reprodutor desktop nativo **Karaokê TV Player** em Electron;
- Validar a sincronização em tempo real entre a fila de pedidos e a tela de reprodução na TV, documentando o progresso atual, os testes preliminares e o roteiro de evolução do MVP.

1.3 JUSTIFICATIVA

A realização do projeto justifica-se em três eixos complementares: a relevância operacional para o estabelecimento atendido, a contribuição para pequenos negócios locais e o cumprimento da função extensionista da instituição, com a formação prática dos estudantes.

Para o estabelecimento parceiro e seus frequentadores, a digitalização do fluxo de pedidos elimina o uso de fichas de papel, impede a repetição de músicas já presentes na fila e dá aos clientes visibilidade sobre sua posição na ordem de apresentações. Ao mesmo tempo, o organizador deixa de pesquisar e digitar manualmente cada vídeo no aplicativo do YouTube do projetor (Capítulo 2), o que libera sua atenção para a condução do palco e o atendimento do salão.

No setor de bares e restaurantes, a informatização avançou sobretudo nas rotinas comerciais. Pesquisa da Associação Brasileira de Bares e Restaurantes (Abrasel) com mais de 1,5 mil empresários indicou que 83% dos estabelecimentos usam algum tipo de software

para controlar as vendas e 75% usam ferramentas de controle financeiro, mas apontou espaço de crescimento em outras áreas da operação, como o controle de reservas (JORNAL DO COMÉRCIO, 2023). No estabelecimento parceiro, a Noite de Karaokê é um exemplo dessas áreas ainda não atendidas por sistemas: a organização dos pedidos e da fila de cantores continua manual.

Entre as micro, pequenas e médias empresas (MPMEs) em geral, a gestão manual ainda é frequente. O *1º Panorama da Gestão de Despesas Corporativas no Brasil*, da Conta Simples e da Visa, realizado com 1.524 empresários entre novembro e dezembro de 2024, mostrou que 39% das MPMEs controlam suas despesas de forma manual, com planilhas e cadernos, percentual que chega a 45% entre as microempresas. Entre os motivos para não adotar sistemas especializados, 44% dos entrevistados citaram o custo percebido como alto (FOLHA DE PERNAMBUCO, 2025). Esse cenário reforça a importância de soluções de baixo custo, como a proposta neste projeto, cujo único custo previsto é o da hospedagem em nuvem (Seção 3.7). A relevância econômica desses negócios também é expressiva: segundo o Sebrae, com base no Caged, as micro e pequenas empresas responderam por sete em cada dez empregos formais gerados no Brasil entre janeiro e novembro de 2025 (BALDO, 2026).

Por fim, o projeto cumpre a função extensionista do IFSP. A Lei nº 11.892/2008, que criou os Institutos Federais, inclui entre suas finalidades desenvolver programas de extensão (art. 6º, VII) e, entre seus objetivos, desenvolver atividades de extensão em articulação com o mundo do trabalho e os segmentos sociais (art. 7º, IV) (BRASIL, 2008). A Resolução CNE/CES nº 7/2018 estabelece que as atividades de extensão devem compor, no mínimo, 10% da carga horária curricular dos cursos de graduação (BRASIL, 2018b), e a disciplina de Projeto Integrado de Extensão, na qual este trabalho se insere, é uma das formas de cumprir essa diretriz. Para os discentes, o projeto proporciona a vivência de todas as etapas da engenharia de software com um cliente real: a elicitação de requisitos, a modelagem UML, o desenvolvimento com React, Spring Boot e Electron, o versionamento colaborativo em Git e a tomada de decisões técnicas diante de restrições concretas, como as limitações de reprodução de vídeos protegidos por direitos autorais e a cota de uso da API do YouTube.

1.4 ESCOPO DO PROJETO

O escopo do projeto abrange a especificação completa da arquitetura e a entrega do **Produto Mínimo Viável (MVP)** do sistema **Karaokê Apache**, contemplando:

1. Levantamento de requisitos, regras de negócio (RN00 a RN04) e modelagem conceitual e lógica do banco de dados (`usuario`, `musica` e `fila`, além da tabela `letra`, prevista no modelo mas fora do escopo do MVP);

2. Interface web responsiva acessível via QR Code no telão para que o usuário pesquise e escolha vídeos musicais diretamente pelo próprio sistema, visualize a música em execução (*Tocando Agora*) e acompanhe a fila numerada;

3. Funcionalidades administrativas de controle de palco (iniciar música, avançar para a próxima faixa da fila, remover item e limpar fila) com bloqueio automático de duplicidade;

4. API Orquestradora em Java 21 com Spring Boot e banco de dados MySQL, hospedados em nuvem, responsáveis pelas regras de negócio, pela pesquisa de vídeos na *YouTube Data API v3* e pela sincronização da fila em tempo real;

5. Aplicativo reprodutor desktop (**Karaokê TV Player**) em Electron, executado no computador do bar e sincronizado em tempo real com a fila, que exibe no telão o vídeo da música da vez e a interface própria do sistema, em duas visões.

1.5 ESTRUTURA DO DOCUMENTO

O restante deste documento está organizado da seguinte forma:

- **Capítulo 2 – Contexto do Projeto:** caracteriza o cenário do parceiro Apache Gru Rock Bar, o estudo de mercado e de concorrentes, o público-alvo, as necessidades identificadas e as regras de negócio conceituais (incluindo o Diagrama de Atividades UML);

- **Capítulo 3 – Análise do Sistema:** apresenta a visão geral da solução, os atores envolvidos, os casos de uso (com Diagrama de Casos de Uso UML), o modelo conceitual do domínio (com Diagrama de Classes Conceitual UML), os requisitos funcionais e não funcionais e as restrições do sistema;

- **Capítulo 4 – Projeto do Sistema:** detalha a arquitetura (com Diagrama de Componentes UML), o projeto dos componentes e a integração com o YouTube, o modelo relacional de dados (com Diagrama de Classes de Projeto UML), o projeto das interfaces de usuário, os diagramas de interação (dois Diagramas de Sequência UML: pedido de música pelo cliente e controle do palco com reprodução no telão), as tecnologias adotadas e os mecanismos de segurança e controle de acesso (com Diagrama de Estados UML do ciclo de vida do pedido);

- **Capítulo 5 – Implementação:** descreve a implementação da solução, distinguindo o que foi validado na Prova de Conceito do que está em implementação no MVP, e aborda o ambiente de desenvolvimento, a estrutura da aplicação, os componentes, a integração dos módulos (com Diagrama de Implantação UML), os testes realizados e planejados, os resultados em relação aos objetivos e as limitações atuais;

- **Capítulo 6 – Conclusão:** retoma os objetivos do projeto, sintetiza as considerações finais sobre o desenvolvimento até o estágio de MVP, relata as dificuldades enfrentadas e aponta os trabalhos futuros;
- **Elementos Pós-Textuais:** reúnem as Referências e o Anexo normativo (LGPD).

2 CONTEXTO DO PROJETO

Este capítulo apresenta o contexto real no qual o projeto **Karaokê Apache** está inserido, incluindo um estudo sobre o segmento de bares com entretenimento musical, índices de digitalização de micro e pequenas empresas no Brasil, análise de ferramentas concorrentes, caracterização do cenário do estabelecimento parceiro, público-alvo, necessidades identificadas e regras de negócio do domínio.

Sistemas de informação permitem às empresas melhorar a eficiência de suas operações e a qualidade do atendimento aos clientes (O'BRIEN; MARAKAS, 2013). No setor de bares e restaurantes, essa informatização já alcança as rotinas comerciais: pesquisa da Abrasel com mais de 1,5 mil empresários indicou que 83% dos estabelecimentos usam algum software para controlar as vendas e 75% usam ferramentas de controle financeiro, com espaço de crescimento em outras áreas da operação (JORNAL DO COMÉRCIO, 2023). Ainda assim, a gestão manual permanece comum entre as micro, pequenas e médias empresas: 39% delas controlam as despesas com planilhas e cadernos, e 44% apontam o custo como motivo para não adotar sistemas especializados (FOLHA DE PERNAMBUCO, 2025). Quando atividades de entretenimento, como noites de karaokê, dependem de controles manuais em papel, o problema operacional chega à experiência do cliente, com pedidos esquecidos, atrasos entre uma apresentação e outra e incerteza sobre a ordem de chamada, como observado no estabelecimento parceiro (Seção 2.1).

Por outro lado, as principais soluções comerciais de karaokê para estabelecimentos têm custo elevado para um pequeno bar brasileiro e não utilizam o acervo do YouTube. Plataformas internacionais como o **Singa Business** (SINGA BUSINESS, 2026) e o **KaraFun Business** (KARAFUN BUSINESS, 2026a) cobram mensalidades em dólar: os planos do KaraFun Stage, voltados a apresentações com cantores, custam de US$ 49,99 a US$ 209,99 por mês, com compromisso de 12 meses (KARAFUN BUSINESS, 2026b), e os do Singa Business partem de US$ 59 por mês (SINGA, 2026). Ambas trabalham com catálogos próprios licenciados, com mais de 90 mil músicas no KaraFun e mais de 140 mil no Singa (KARAFUN BUSINESS, 2026a; SINGA, 2026), o que impede o uso de versões de karaokê disponíveis no YouTube. Já o **Juks** apresenta-se como uma *jukebox* digital para bares e restaurantes (JUKS, 2026), voltada à execução de músicas no ambiente, e não à organização de apresentações de karaokê. A Tabela 1 sintetiza o estudo comparativo realizado pela equipe.

Tabela 1 – Estudo comparativo entre soluções existentes no mercado e o sistema proposto.

Critério / Recurso Analisado	**Singa Business**	**KaraFun Business**	**Juks**	**Karaokê Apache**
Pedidos pelo celular via QR Code no telão	Parcial	Sim	Sim	Sim
Escolha de vídeos pelo próprio sistema	Sim	Sim	Sim	Sim
Fila de cantores com moderação de palco	Sim	Sim	Não	Sim
Reprodutor dedicado para o telão (TV)	Sim	Sim	Parcial	Sim (Electron)
Suporte ao acervo de karaokê do YouTube	Não	Não	Parcial	Sim
Bloqueio automático de música duplicada	Parcial	Parcial	Sim	Sim
Custo para o bar parceiro	Alto (US$)	Alto (US$)	Médio (R$)	Baixo (somente hospedagem)

Fonte: Elaborado pelos autores a partir de KARAFUN BUSINESS (2026b), SINGA (2026) e JUKS (2026).

2.1 CENÁRIO

O parceiro demandante do projeto é o estabelecimento comercial **Apache Gru Rock Bar**, registrado sob a razão social `APACHE GRU ROCK BAR LTDA` e inscrito no CNPJ sob o nº `64.687.141/0001-07`. O bar está localizado na Avenida Torres Tibagy, nº 58, no bairro Gopouva, em Guarulhos – SP (CEP 07062-000), e sua atividade principal é classificada no CNAE 5611-2/05, que corresponde a bares e outros estabelecimentos especializados em servir bebidas, com entretenimento (CNPJ API, 2026).

O Apache Gru é um bar de rock que recebe apresentações de bandas, como os shows do tributo *Roots – Sepultura Tributo* realizados em agosto de 2025 e janeiro de 2026 (SETLIST.FM, 2026). Às quintas-feiras, a partir das 20h, o bar realiza a sua **Noite de Karaokê**, sem cobrança para cantar (MOTOFEST BRASIL, 2026). No cenário atual de operação do salão, é utilizado um projetor *whitelabel* com conexão à internet, sendo diretamente por ele acessado o aplicativo do YouTube e colocados os vídeos manualmente a cada apresentação.

Dessa forma, a dinâmica da Noite de Karaokê ocorria de maneira inteiramente manual: os clientes precisavam solicitar suas músicas verbalmente ou em anotações de papel, e o organizador precisava digitar e pesquisar manualmente cada vídeo no YouTube diretamente pela interface do projetor *whitelabel* conectado à internet, interrompendo a fluidez entre uma apresentação e outra e dificultando o controle da fila de espera.

2.2 PÚBLICO-ALVO

O projeto atende diretamente três perfis de público-alvo envolvidos na operação da Noite de Karaokê do Apache Gru Rock Bar:

1. **Frequentadores e Cantores Amadores (Clientes do Bar):** jovens e adultos frequentadores do Apache Gru Rock Bar que comparecem individualmente ou em grupos de amigos. Possuem familiaridade cotidiana com smartphones e aplicativos web, mas buscam uma interação rápida e intuitiva em ambiente noturno (baixa luminosidade), sem desejar baixar aplicativos pesados apenas para pedir uma música;

2. **Organizador / Operador de Palco e Som (Administrador Operacional):** colaborador responsável por conduzir a noite de karaokê, chamar o próximo cantor no microfone, controlar o equipamento de áudio e garantir que o telão exiba continuamente a música correta sem interrupções técnicas;

3. **Gestores e Proprietários do Estabelecimento:** responsáveis pela administração do Apache Gru Rock Bar, interessados em aumentar a satisfação e o tempo de permanência dos clientes no salão, reduzir a sobrecarga da equipe de atendimento e conhecer o repertório mais popular entre seus frequentadores.

2.3 NECESSIDADES IDENTIFICADAS

A partir do acompanhamento do cenário operacional do Apache Gru Rock Bar e das conversas com os envolvidos, foram identificadas quatro necessidades fundamentais da comunidade atendida:

- **N01 – Autonomia para escolha de vídeos pelo próprio sistema:** necessidade de permitir que o cliente acesse a plataforma via QR Code exibido no telão e escolha o vídeo musical desejado diretamente pelo próprio sistema em seu celular, eliminando o uso de fichas de papel;

- **N02 – Transparência e previsibilidade na fila de espera:** necessidade de dar visibilidade pública e em tempo real para todos os participantes sobre qual música está tocando no momento, quantas pessoas estão na fila e qual é a ordem das próximas apresentações;

- **N03 – Prevenção de repetições e garantia de rotatividade justa:** necessidade de evitar que uma mesma canção seja cantada repetidas vezes em curto espaço de tempo ou que um único participante monopolize a fila;

- **N04 – Automação e limpeza visual da reprodução no telão:** necessidade de substituir a busca manual direta no projetor *whitelabel* por um reprodutor dedicado, controlado remotamente a partir da fila, que exiba no telão o vídeo da música da vez sem expor telas de busca ou menus intermediários.

2.4 REGRAS DE NEGÓCIO

Para atender às necessidades identificadas respeitando as políticas operacionais do estabelecimento, foram estabelecidas cinco Regras de Negócio (RN00 a RN04) em nível conceitual, documentadas na planilha de engenharia do projeto (*Regras de Negócio / Requisitos – Separados por Componentes*) e descritas na Tabela 2.

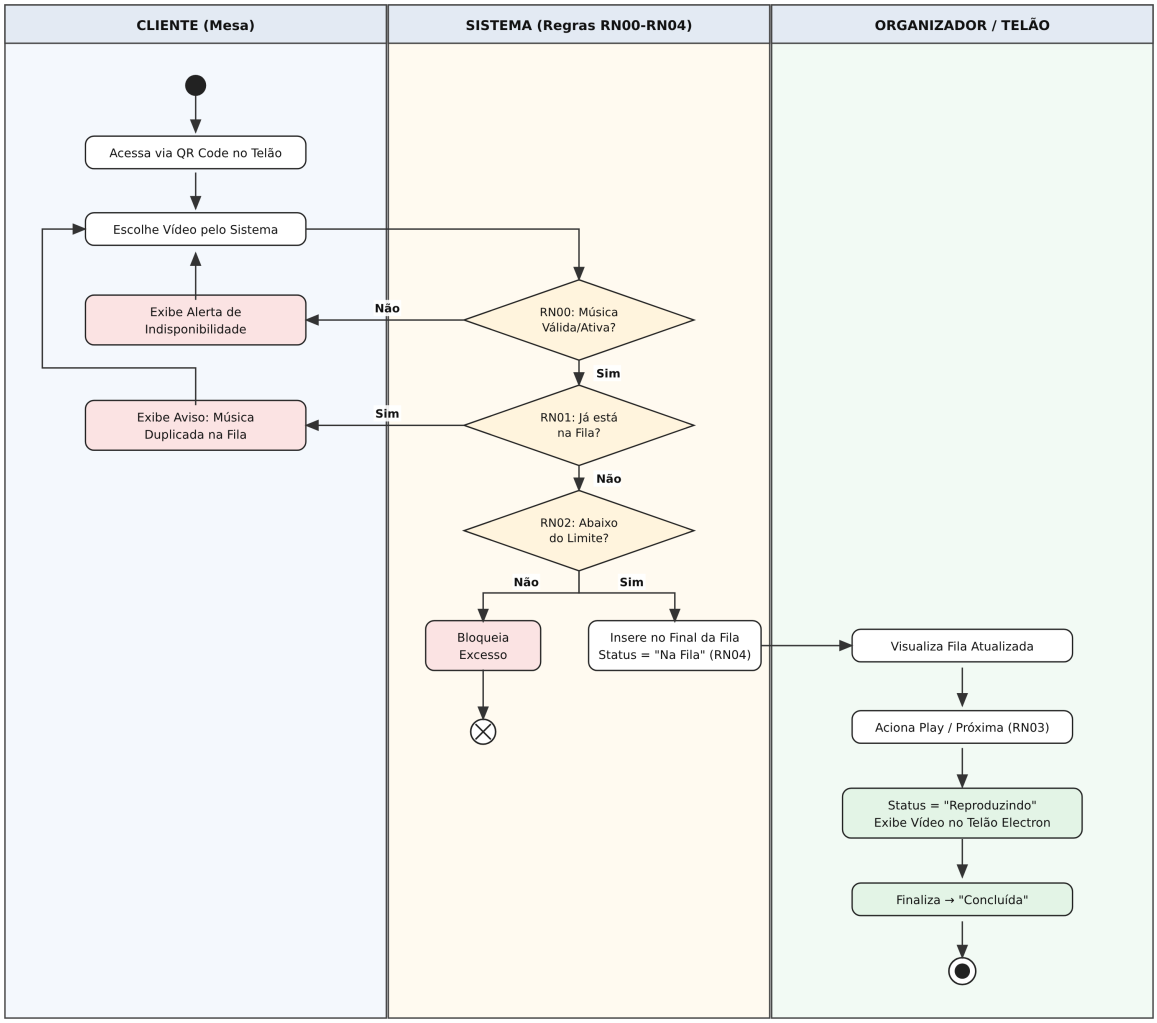
Tabela 2 – Regras de negócio do domínio do sistema Karaokê Apache.

Código	**Regra de Negócio**	**Descrição Conceitual e Política de Domínio**
RN00	Validação de Disponibilidade da Música	Toda solicitação musical deve referenciar um item disponível no catálogo ou uma fonte audiovisual válida e ativa antes de ingressar na fila de apresentações.
RN01	Proibição de Música Duplicada na Fila Ativa	Uma música que já se encontra aguardando execução na fila ativa ou que está sendo reproduzida no momento não pode ser novamente inserida na fila até a conclusão de sua execução atual.
RN02	Limite de Pedidos Simultâneos por Participante	Para preservar a rotatividade entre as mesas do bar, cada cliente autenticado possui um limite configurável de pedidos ativos simultâneos na fila (padrão de 1 a 2 músicas pendentes por vez).
RN03	Exclusividade Administrativa do Controle de Palco	O acionamento de início da reprodução, o avanço para a próxima faixa e a remoção de itens da fila (inclusive por ausência do cantor) são prerrogativas exclusivas do organizador (Administrador). A ordem da fila segue a ordem de chegada dos pedidos (FIFO) e não é alterada manualmente.
RN04	Ciclo de Vida e Rastreabilidade do Pedido	Todo pedido aceito percorre um fluxo controlado de estados (*Na Fila*, *Reproduzindo*, *Concluído* ou *Cancelado*), mantendo o registro do solicitante e do horário para controle da sessão.

Fonte: Elaborado pelos autores a partir da planilha de engenharia do projeto (2026).

A Figura 1 apresenta o **Diagrama de Atividades UML** que modela o fluxo operacional de negócio desde a solicitação da música pelo cliente na mesa até a conclusão da apresentação no palco, destacando os pontos de decisão regidos pelas regras RN00, RN01, RN02 e RN03.

Figura 1 – Diagrama de Atividades UML – Fluxo de Pedido Musical e Execução na Fila



Fonte: Elaborado pelos autores (2026).

3 ANÁLISE DO SISTEMA

Este capítulo transforma a compreensão do contexto operacional do Apache Gru Rock Bar e das necessidades identificadas em uma visão estruturada do sistema **Karaokê Apache**, definindo sua visão geral, os atores envolvidos, os casos de uso, o modelo conceitual de domínio, os requisitos funcionais e não funcionais e as restrições do sistema.

3.1 VISÃO GERAL DO SISTEMA

O sistema **Karaokê Apache** foi concebido como uma plataforma distribuída centrada na orquestração da fila de reprodução musical. Em nível macroscópico, o sistema atua como uma ponte automatizada entre o salão do bar (onde os clientes realizam pedidos e acompanham a fila pelos seus smartphones) e o palco (onde o administrador coordena a ordem de apresentações e o terminal reprodutor exibe os vídeos de karaokê na TV).

Utilizando a abordagem de *Feature-Driven Development* (FDD) adotada pela equipe na fase de análise, as capacidades macroscópicas do sistema organizam-se em três grandes funcionalidades (*Features*) centrais:

- **FT-001 – Solicitar música para a fila do karaokê:** permite ao participante autenticado pesquisar e escolher um vídeo musical diretamente pelo próprio sistema, passando pelas validações automáticas de disponibilidade e anti-duplicidade (RN00, RN01 e RN02);
- **FT-002 – Gerenciar e moderar a fila de reprodução:** permite ao organizador visualizar a fila ordenada por chegada (FIFO), iniciar a reprodução do primeiro item, avançar para a próxima música ou remover pedidos inválidos (RN03 e RN04);
- **FT-003 – Sincronizar e reproduzir faixa no telão:** atualiza instantaneamente o estado da música ativa para que o reprodutor desktop dedicado renderize o vídeo correspondente em tela cheia na TV do bar.

3.2 ATORES ENVOLVIDOS

Foram identificados três atores que interagem diretamente com as fronteiras do sistema, cujos papéis são detalhados na Tabela 3.

Tabela 3 – Identificação e caracterização dos atores do sistema Karaokê Apache.

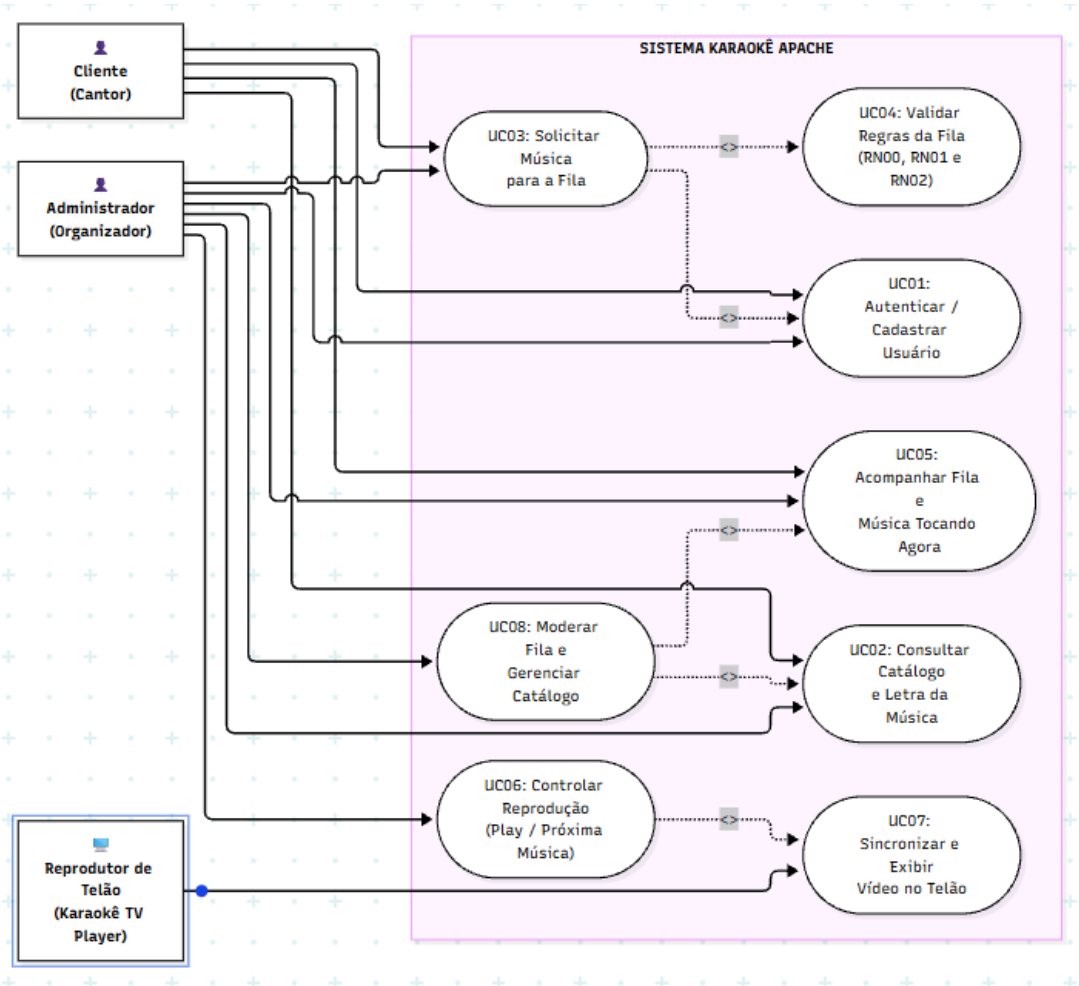
Ator	Tipo de Ator	Papel e Interações com o Sistema
Cliente (Cantor)	Humano (Primário)	Frequentador do Apache Gru Rock Bar que acessa a interface móvel via QR Code no telão para se identificar, pesquisar e escolher vídeos pelo sistema, adicionar pedidos à fila e consultar sua posição de espera.
Administrador (Organizador)	Humano (Primário)	Operador responsável pela condução da Noite de Karaokê. Possui permissão exclusiva para iniciar faixas, avançar para a próxima música, remover itens, limpar a fila e gerenciar o catálogo.
Reprodutor de Telão (*Karaokê TV Player*)	Sistema / Terminal (Secundário)	Instância desktop em Electron executada no computador conectado à TV do palco, que consome o estado da faixa ativa e renderiza o fluxo de áudio e vídeo do YouTube em tela cheia.

Fonte: Elaborado pelos autores (2026).

3.3 CASOS DE USO

A partir dos atores identificados, foram levantados oito Casos de Uso (UC01 a UC08) que especificam o comportamento esperado do sistema sob a perspectiva de seus usuários. A Figura 2 apresenta o **Diagrama de Casos de Uso UML** do sistema, seguido pelo detalhamento descritivo na Tabela 4.

Figura 2 – Diagrama de Casos de Uso UML – Sistema Karaokê Apache



Fonte: Elaborado pelos autores (2026).

Tabela 4 – Especificação dos casos de uso (UC) do sistema Karaokê Apache.

Código	**Nome do Caso de Uso**	**Atores**	**Resumo do Fluxo Principal e Comportamento Esperado**
UC01	Autenticar / Cadastrar Usuário	Cliente, Administrador	O usuário informa suas credenciais (ou realiza cadastro rápido com nome e e-mail) para acessar as funcionalidades correspondentes ao seu perfil.
UC02	Pesquisar e Escolher Vídeo no Sistema	Cliente, Administrador	O ator pesquisa vídeos de karaokê disponíveis por título ou artista diretamente na interface do próprio sistema.
UC03	Solicitar Música para a Fila	Cliente, Administrador	O ator escolhe o vídeo musical diretamente pelo sistema; a aplicação executa as validações do caso de uso UC04 e adiciona o item ao final da fila.
UC04	Validar Regras da Fila	Sistema (*Include* de UC03)	Verifica se o vídeo é válido/disponível (RN00), se não está duplicado na fila ativa (RN01) e se o cliente respeita o limite de pedidos simultâneos (RN02).
UC05	Acompanhar Fila e Música Atual	Cliente, Administrador	Exibe em tempo real o card *Tocando Agora* e a lista ordenada de próximas músicas com a quantidade total na fila.
UC06	Controlar Reprodução (Play / Próxima)	Administrador	O administrador inicia uma música da fila ou aciona o botão *Próxima música*, promovendo o próximo item da fila para o estado *Reproduzindo*.
UC07	Sincronizar e Exibir Vídeo no Telão	Administrador, Reprodutor TV	A alteração da faixa ativa dispara a atualização imediata da janela do *Karaokê TV Player* (Electron), que carrega o vídeo em tela cheia.
UC08	Moderar Fila e Gerenciar Catálogo	Administrador	Permite excluir pedidos individuais, limpar a fila ao encerrar a noite e cadastrar ou inativar músicas no banco de dados.

Fonte: Elaborado pelos autores (2026).

3.4 MODELO CONCEITUAL

O modelo conceitual do domínio foi originalmente construído pela equipe na ferramenta **brModelo** (arquivo `Conceitual.brM3` armazenado no acervo do projeto) e traduzido para o **Diagrama de Classes Conceitual UML** apresentado na Figura 3.

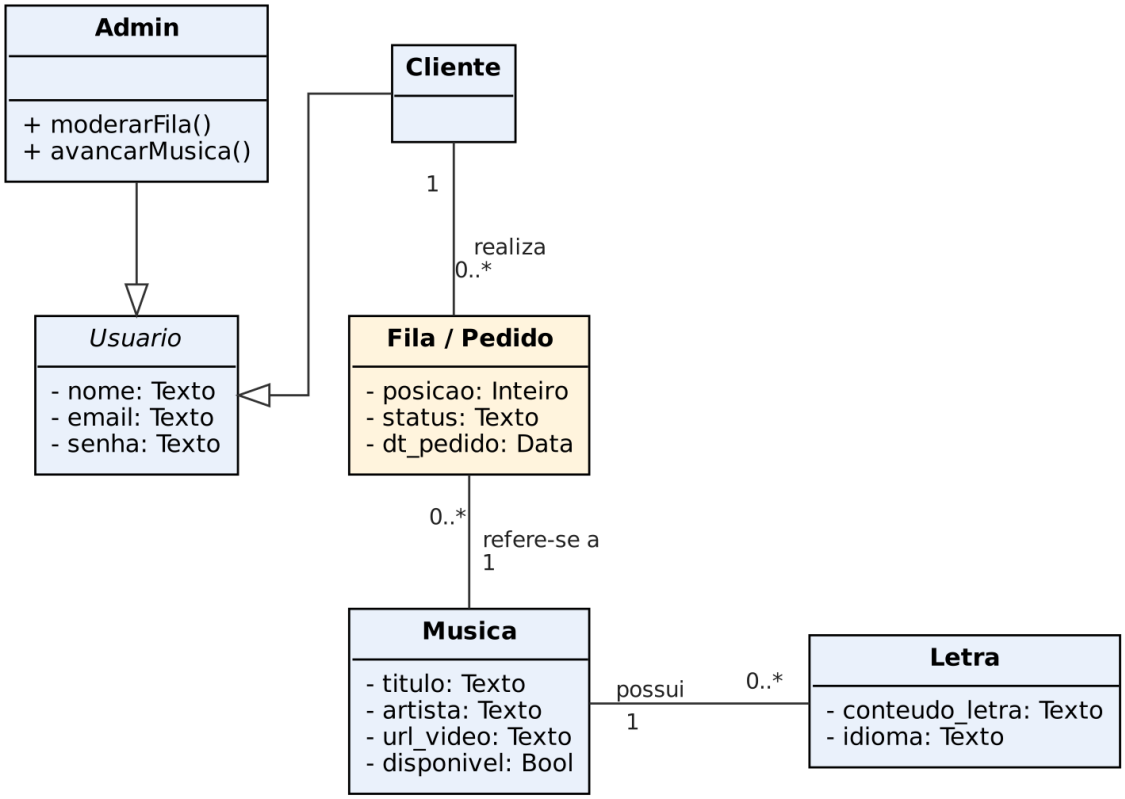
O domínio é estruturado em torno de quatro conceitos essenciais, independentes de decisões de implementação:

- **`Usuario` (com generalização/especialização em `Admin` e `Cliente`):** representa qualquer pessoa cadastrada que interage com o sistema, contendo os atributos conceituais `nome`, `email` e `senha`. A especialização distingue o `Cliente` (autor dos pedidos musicais) do `Admin` (responsável pela moderação);
- **`Musica`:** representa a obra musical disponível para interpretação no karaokê,

caracterizada por `titulo`, `artista`, `url_video` e estado de disponibilidade (`disponivel`);

- **`Letra`:** representa o texto da canção (`conteudo_letra`, `idioma`) vinculado à classe `Musica` pela associação *Possui* com multiplicidade 1 para 0..∗ (estrutura de dados já prevista no modelo para suportar a funcionalidade de letras na próxima etapa evolutiva do projeto);
- **`Fila` (Classe Associativa `Adiciona`):** materializa o vínculo temporal e ordenado entre um `Cliente` (0..∗) e uma `Musica` (0..∗), registrando a `posicao` na fila, o `status` do pedido e a data/hora da solicitação (`dt_pedido`).

Figura 3 – Diagrama de Classes Conceitual UML (Derivado de `Conceitual.brM3`)



Fonte: Adaptado de `Conceitual.brM3` elaborado pelos autores (2026).

3.5 REQUISITOS FUNCIONAIS

A Tabela 5 especifica de forma objetiva os Requisitos Funcionais (RF01 a RF12) que o sistema deve oferecer para atender às necessidades do Apache Gru Rock Bar.

Tabela 5 – Requisitos funcionais (RF) do sistema Karaokê Apache.

Código	Requisito Funcional	Descrição do Comportamento	Prioridade
RF01	Cadastro e Autenticação de Usuário	Permitir o cadastro e login de usuários diferenciando os perfis de acesso *Cliente* e *Administrador*.	Média
RF02	Pesquisa de Vídeos no Próprio Sistema	Permitir que o usuário pesquise vídeos de karaokê por título, música ou artista diretamente na interface do sistema.	Alta
RF03	Seleção de Vídeo pelo Sistema	Permitir que o usuário escolha o vídeo desejado a partir dos resultados exibidos pelo próprio sistema, sem necessidade de copiar links externos.	Alta
RF04	Adição de Pedido à Fila	Permitir inserir o vídeo escolhido na fila ativa, registrando a identificação da mídia, o título de exibição e a ordem de chegada.	Alta
RF05	Validação de Disponibilidade e Anti-Duplicidade	Validar o identificador do vídeo escolhido (RN00) e bloquear músicas que já estejam presentes na fila ativa ou em reprodução (RN01).	Alta
RF06	Visualização da Fila e Faixa Atual	Exibir em tempo real o destaque *Tocando Agora* e a lista numerada de próximas músicas na fila.	Alta
RF07	Acionamento Direto de Música (*Play*)	Permitir ao administrador iniciar com um clique a reprodução do primeiro pedido da fila, respeitando a ordem de chegada (FIFO).	Alta
RF08	Avanço Automático (*Próxima Música*)	Permitir ao administrador encerrar a faixa atual e promover automaticamente o próximo item da fila (FIFO) para execução.	Alta
RF09	Remoção Individual e Limpeza da Fila	Permitir ao administrador excluir um pedido específico ou limpar toda a fila ao final do evento.	Alta
RF10	Reprodução Dedicada no Telão (Electron)	Carregar e reproduzir automaticamente o vídeo da música ativa na janela nativa do *Karaokê TV Player*.	Alta
RF11	Gestão Administrativa do Catálogo	Permitir cadastrar novas músicas no banco de dados e alterar seu status de disponibilidade.	Média
RF12	Feedback Visual de Estado e Conexão	Exibir indicadores claros de conexão (*Conectado* – *Tocando*) e alertas de validação na interface.	Alta

Fonte: Elaborado pelos autores (2026).

3.6 REQUISITOS NÃO FUNCIONAIS

A Tabela 6 apresenta os Requisitos Não Funcionais (RNF01 a RNF08) que estabelecem os padrões de usabilidade, desempenho, compatibilidade, segurança e privacidade do sistema.

Tabela 6 – Requisitos não funcionais (RNF) do sistema Karaokê Apache.

Código	Categoria	Requisito	Critério de Qualidade exigido
RNF01	Usabilidade	Interface Responsiva e Tema Escuro	A interface web deve adaptar-se a telas móveis e utilizar paleta escura de alto contraste adequada à iluminação noturna do bar.
RNF02	Desempenho	Sincronização em Tempo Real	A troca de música acionada na interface de controle deve refletir na janela do reprodutor Electron em menos de 2 segundos, em condições normais de conexão com a internet.
RNF03	Compatibilidade	Acesso Web Sem Instalação (*Zero-Install*)	Os clientes devem acessar o sistema diretamente por navegadores padrões (Chrome, Safari, Firefox) via leitura do QR Code exibido no telão.
RNF04	Segurança	Isolamento de Contexto no Electron	O reprodutor desktop deve executar com isolamento de contexto ativo e sem exposição direta de APIs críticas do sistema operacional a scripts remotos.
RNF05	Segurança	Proteção de Credenciais e Sessão	As senhas devem ser armazenadas sob hash criptográfico (Bcrypt) e as rotas administrativas protegidas por controle de perfil.
RNF06	Confiabilidade	Consistência de Estado da Fila	A API em Java 21 com Spring Boot e o banco de dados MySQL devem garantir integridade transacional na ordenação e transição de status dos pedidos.
RNF07	Manutenibilidade	Modularidade por Componentes	O código deve manter separação clara entre as responsabilidades de Usuários, Fila, Pedido de Música e Reprodutor.
RNF08	Legal / Privacidade	Conformidade com a LGPD	A coleta de dados dos clientes deve limitar-se ao mínimo necessário para identificação na fila (Art. 6º da Lei nº 13.709/2018).

Fonte: Elaborado pelos autores (2026).

3.7 RESTRIÇÕES DO SISTEMA

O desenvolvimento e a operação do sistema estão sujeitos às seguintes restrições técnicas, operacionais, legais e orçamentárias:

- **Cota de Uso da API de Pesquisa do YouTube:** a pesquisa de vídeos depende da *YouTube Data API v3*, cuja cota padrão é de 10.000 unidades por dia, com cada pesquisa consumindo 100 unidades (GOOGLE, 2026); isso limita o sistema a cerca de 100 pesquisas diárias, o que exige o uso do catálogo local e de *cache* para reduzir as consultas ao serviço;
- **Dependência de Conectividade com a Internet:** como os fluxos de vídeo do MVP são carregados sob demanda a partir do YouTube, o computador de palco do Apache Gru Rock Bar necessita de conexão ativa à internet durante a Noite de Karaokê;

- **Restrição Orçamentária (Baixo Custo):** por tratar-se de um projeto acadêmico de extensão voltado a uma microempresa, a solução deve utilizar exclusivamente ferramentas, bibliotecas e bancos de dados gratuitos ou de código aberto (*open source*), sem custo de licenciamento, e reaproveitar o hardware (computador, TV e som) já existente no bar. O único custo previsto é o da plataforma de hospedagem em nuvem da aplicação web, da API e do banco de dados, que deve ser baixo e compatível com o porte do estabelecimento;
- **Restrição Temporal e Acadêmica:** o cronograma de desenvolvimento é delimitado pelo semestre letivo da disciplina de Projeto Integrado de Extensão do IFSP, razão pela qual a entrega concentra-se na consolidação e validação do Produto Mínimo Viável (MVP);
- **Restrições Legais (LGPD):** o sistema não pode coletar dados pessoais sensíveis nem compartilhar cadastros de frequentadores com terceiros sem consentimento expresso (BRASIL, 2018a).

4 PROJETO DO SISTEMA

Este capítulo estabelece a especificação técnica da solução **Karaokê Apache**, transformando o modelo conceitual e os requisitos da análise em uma definição detalhada da arquitetura distribuída, dos componentes de software, do modelo lógico e físico de dados, do projeto das interfaces de usuário, das interações dinâmicas, das tecnologias adotadas e dos mecanismos de segurança.

4.1 ARQUITETURA DO SISTEMA

A solução **Karaokê Apache** é estruturada em quatro subsistemas: **Cliente Web**, **Admin Web**, **API Orquestradora** (conectada ao **Banco de Dados Relacional**) e o reprodutor desktop **Electron (Karaokê TV Player)**. Além deles, o sistema depende de um serviço externo, a *YouTube Data API v3*, usada na pesquisa de vídeos (GOOGLE, 2026). A reprodução não utiliza API do YouTube: o Electron abre diretamente o link do vídeo registrado no item da fila.

A organização segue o estilo cliente-servidor descrito por Sommerville (SOMMERVILLE, 2019): as interfaces web e o Electron são clientes que acessam serviços oferecidos por um servidor central, a API Orquestradora. Nessa API, as regras de negócio ficam concentradas em uma camada de serviço, no sentido do padrão *Service Layer* de Fowler (FOWLER, 2006), de modo que nenhum cliente acessa o banco de dados diretamente. A divisão resultante separa duas responsabilidades:

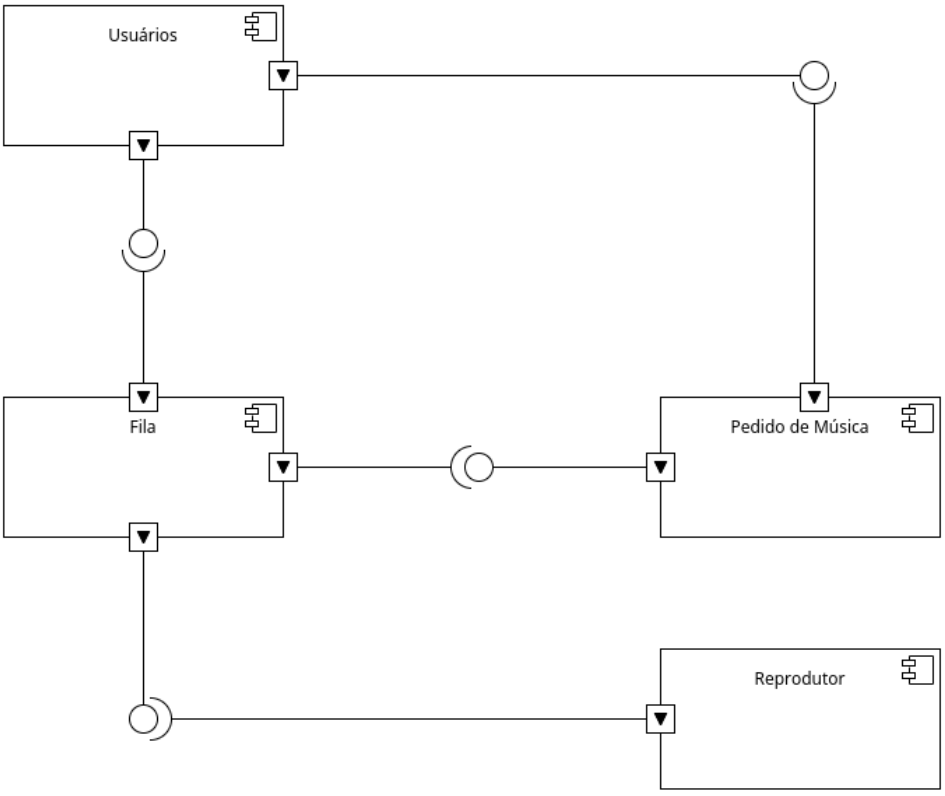
- **Gestão do karaokê (subsistemas web e API): Cliente Web**, **Admin Web** e **API Orquestradora** reúnem o que envolve a interação das pessoas com o sistema: o cadastro e a autenticação de usuários, a pesquisa e o pedido de músicas e o gerenciamento da fila (ordenação, remoção, limpeza e definição da faixa ativa). A API, apoiada pelo banco de dados relacional, mantém o estado oficial da fila;
- **Reprodução no telão (subsistema Electron):** o aplicativo desktop **Karaokê TV Player** é responsável por **buscar as músicas que estão na fila** e **reproduzi-las** na TV do bar. Ele não realiza cadastros, não registra pedidos e não decide a ordem da fila. Sua única informação de retorno é o aviso de término do vídeo, a partir do qual a API conclui o pedido e promove o próximo item (RN04).

A janela do Electron exibida no telão é dividida em **duas visões**. A primeira carrega o vídeo do YouTube a partir do link registrado no item da fila e ocupa uma parte da tela. A segunda, no restante da tela, apresenta a interface própria do sistema. O vídeo é aberto pelo seu link, e não incorporado a uma página por meio do *YouTube IFrame Player API*, porque no contexto do karaokê muitos vídeos apresentam falhas de reprodução

quando incorporados, em razão de restrições de direitos autorais definidas pelos detentores do conteúdo.

No nível lógico, essa divisão é expressa em quatro componentes (**Usuários**, **Pedido de Música**, **Fila** e **Reprodutor**), representados no **Diagrama de Componentes UML** da Figura 4. Os três primeiros pertencem aos subsistemas de gestão (web e API), e o **Reprodutor** corresponde ao subsistema Electron. Os dois lados se comunicam apenas pelas interfaces fornecidas pelo componente **Fila**. A distribuição física dos subsistemas nos equipamentos do bar é apresentada no Diagrama de Implantação do Capítulo 5 (Figura 10).

Figura 4 – Diagrama de Componentes UML



Fonte: Elaborado pelos autores no software UMLet (2026).

4.2 PROJETO DOS COMPONENTES

Seguindo a orientação de Pressman e Maxim (PRESSMAN; MAXIM, 2016) para o projeto em nível de componente, cada componente reúne responsabilidades de um mesmo assunto e se comunica com os demais apenas por interfaces declaradas. A Tabela 7 relaciona cada subsistema aos componentes que ele materializa: o Electron abriga apenas o

componente **Reprodutor**, e os demais componentes estão distribuídos entre as interfaces web e a API.

Tabela 7 – Correspondência entre subsistemas e componentes lógicos.

Subsistema	**Componentes**	**Papel na Arquitetura**
Cliente Web	`Usuários`, `Pedido de Música`, `Fila` (consulta)	Interface do frequentador: cadastro e login, busca e pedido de músicas e acompanhamento da sua posição na fila.
Admin Web	`Usuários`, `Fila` (gerenciamento)	Interface do organizador do palco: autenticação como `admin` e gerenciamento da fila (*Play*, *Próxima*, *Remover* e *Limpar*).
API Orquestradora + Banco de Dados	`Usuários`, `Pedido de Música`, `Fila` (regras e persistência)	Aplica as regras de negócio (RN00–RN04), persiste usuários, músicas e pedidos e mantém o estado oficial da fila e da faixa ativa.
Electron (*Karaokê TV Player*)	`Reprodutor`	Busca na fila a música a ser executada, exibe no telão o vídeo (por meio do link) ao lado da interface própria do sistema e avisa a API quando o vídeo termina.

Fonte: Elaborado pelos autores (2026).

A Tabela 8 detalha as responsabilidades de cada componente e nomeia as interfaces fornecidas e requeridas, inclusive a interface com o serviço externo de pesquisa do YouTube.

Tabela 8 – Detalhamento técnico dos componentes arquiteturais e suas interfaces.

Componente	**Interfaces (Fornecidas / Requeridas)**	**Estrutura Interna e Responsabilidades**
`Usuários` (Web + API)	Fornece: `IAutenticacao`, `IPerfilUsuario`	Responsável pelo cadastro, autenticação e diferenciação de permissões entre os papéis `cliente` (frequentador da mesa) e `admin` (organizador do palco).
`Pedido de Música` (Web + API)	Requer: `IAutenticacao`, `IBuscaVideos` (externa); Fornece: `IPedidoMusica`	Responsável pela pesquisa e escolha de vídeos musicais diretamente pelo sistema (RF02, RF03), pela identificação do vídeo selecionado e pela validação de disponibilidade (RN00). A pesquisa é feita pela API, que consulta a *YouTube Data API v3* por meio de `IBuscaVideos`.
`Fila` (Web + API)	Requer: `IAutenticacao`, `IPedidoMusica`; Fornece: `IControleFila`, `IEventoReproducao`	Gerencia a lista ordenada de pedidos (`queue`) e o item em execução (`currentSong`), aplica o bloqueio de duplicidade (RN01) e as transições de estado do pedido (RN04). `IControleFila` reúne as operações de consulta (faixa ativa e próximos itens) e as de gerenciamento (*Play*, *Próxima*, *Remover* e *Limpar*), estas restritas ao perfil `admin` (RN03). `IEventoReproducao` recebe o aviso de término do vídeo e marca o pedido como concluído.
`Reprodutor` (Electron)	Requer: `IControleFila` (somente consulta), `IEventoReproducao`	Único componente do aplicativo desktop *Karaokê TV Player*: busca na `Fila` a música que deve ser executada (`currentSong`) e o link do vídeo correspondente. Organiza a janela do telão em duas visões: o vídeo do YouTube, carregado diretamente pelo link, ocupa uma parte da tela, e a interface própria do sistema ocupa o restante. Ao identificar o término do vídeo, comunica-o por `IEventoReproducao`. Não realiza cadastros, pedidos nem alterações na ordem dos itens.

Fonte: Elaborado pelos autores (2026).

O fluxo de pedido funciona da seguinte forma: o cliente digita o termo de pesquisa na interface web, e a API consulta a *YouTube Data API v3*, que devolve, para cada resultado, o identificador do vídeo (`videoId`) e dados de exibição, como o título. Ao clicar na música desejada, o cliente a adiciona à fila. A API registra a música no catálogo local, caso ainda não exista, e insere o pedido na fila referenciando-a (Seção 4.3). Posteriormente, o **Reprodutor** monta o link do vídeo a partir do identificador da música e o abre no telão.

A integração com o YouTube impõe duas decisões de projeto ao componente **Pedido de Música**. A primeira é que a chave de acesso da *YouTube Data API v3* fica armazenada **somente no back-end**: as interfaces web enviam o termo de pesquisa à API Orquestradora, que consulta o YouTube e devolve os resultados, sem expor a chave aos navegadores. A segunda decorre da cota de uso do serviço: um projeto recebe, por padrão, 10.000 unidades por dia, e cada chamada de pesquisa (`search.list`) consome 100 unidades (GOOGLE, 2026), o que limita o bar a cerca de 100 pesquisas diárias. Para

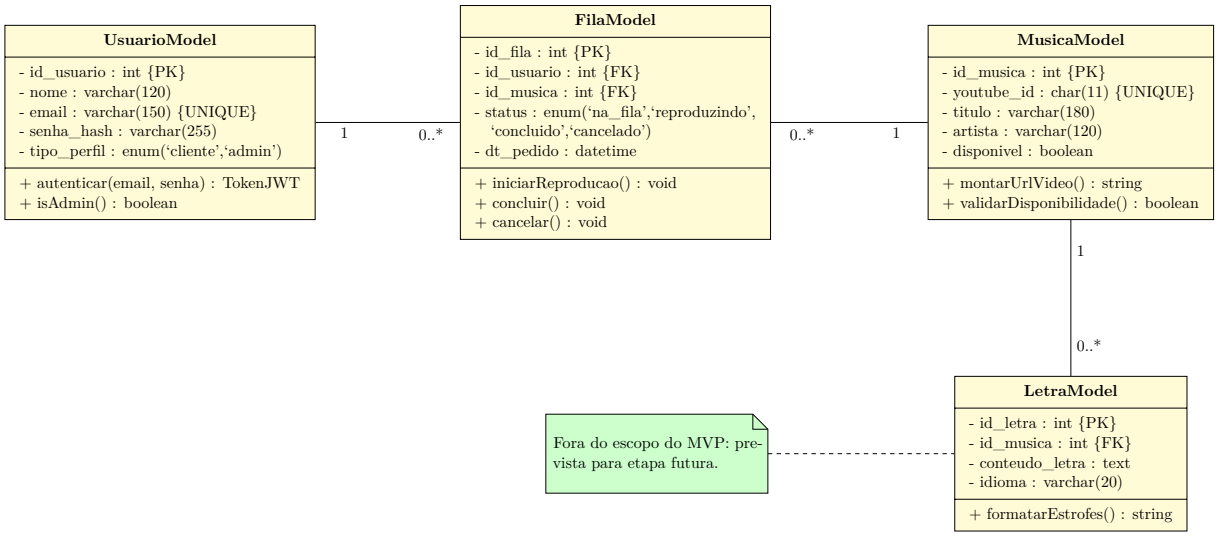
reduzir esse consumo, a API consulta primeiro o catálogo local (tabela `musica`) e mantém um *cache* dos resultados de pesquisas recentes, recorrendo ao YouTube apenas quando a música não é encontrada.

O aviso de término enviado pelo Reprodutor corresponde à transição *Reproduzindo* → *Concluído* do ciclo de vida do pedido (RN04). No estágio atual do MVP, o avanço para a próxima música ainda é acionado manualmente pelo administrador, e o avanço automático por esse evento está previsto como incremento seguinte (Capítulo 5).

4.3 MODELO DE DADOS

O projeto de dados converte o modelo conceitual em uma estrutura relacional na terceira forma normal (3FN), acrescentando chaves primárias artificiais (`PK`), chaves estrangeiras (`FK`), tipos físicos de dados, restrições de integridade e métodos de negócio das classes de domínio. O modelo é composto por quatro tabelas: `usuario`, `musica` e `fila`, que compõem o escopo do MVP, e `letra`, que já integra o modelo, mas **não faz parte do escopo do MVP**. A Figura 5 apresenta o **Diagrama de Classes de Projeto UML**, e a Tabela 9 documenta o Dicionário de Dados relacional.

Figura 5 – Diagrama de Classes de Projeto UML



Fonte: Elaborado pelos autores (2026).

Tabela 9 – Dicionário de dados do modelo relacional do sistema Karaokê Apache.

Atributo	Tipo Físico	Restrições	Descrição no Projeto
Tabela `usuario`			
`id_usuario`	`INT`	PK, AI	Chave primária do usuário.
`nome`	`VARCHAR(120)`	`NOT NULL`	Nome do participante ou operador.
`email`	`VARCHAR(150)`	`NOT NULL`, `UNIQUE`	E-mail de login do usuário.
`senha_hash`	`VARCHAR(255)`	`NOT NULL`	Senha criptografada (*bcrypt*).
`tipo_perfil`	`ENUM('cliente', 'admin')`	`NOT NULL`, `DEFAULT 'cliente'`	Perfil de permissão no sistema.
Tabela `musica`			
`id_musica`	`INT`	PK, AI	Identificador único da música.
`youtube_id`	`CHAR(11)`	`NOT NULL`, `UNIQUE`	Identificador do vídeo no YouTube (`videoId`), com 11 caracteres. O link do vídeo é montado a partir dele.
`titulo`	`VARCHAR(180)`	`NOT NULL`	Título da música, obtido do resultado da pesquisa ou do cadastro administrativo.
`artista`	`VARCHAR(120)`	`NULL`	Banda ou intérprete, preenchido quando conhecido (por exemplo, no cadastro administrativo – RF11).
`disponivel`	`BOOLEAN`	`NOT NULL`, `DEFAULT TRUE`	Indica se a música pode ser pedida (RN00).
Tabela `fila`			
`id_fila`	`INT`	PK, AI	Identificador único do pedido na fila.
`id_usuario`	`INT`	FK → `usuario`, `NOT NULL`	Cliente que fez o pedido.
`id_musica`	`INT`	FK → `musica`, `NOT NULL`	Música pedida.
`status`	`ENUM('na_fila', 'reproduzindo', 'concluido', 'cancelado')`	`NOT NULL`, `DEFAULT 'na_fila'`	Estado atual do pedido (RN04).
`dt_pedido`	`DATETIME`	`NOT NULL`, `DEFAULT CURRENT_TIMESTAMP`	Data e hora do pedido; define a ordem de chamada (FIFO).
Tabela `letra` (fora do escopo do MVP)			
`id_letra`	`INT`	PK, AI	Identificador único da letra.
`id_musica`	`INT`	FK → `musica`, `NOT NULL`	Música à qual a letra pertence.
`conteudo_letra`	`TEXT`	`NOT NULL`	Letra completa da canção.
`idioma`	`VARCHAR(20)`	`NOT NULL`, `DEFAULT 'pt-BR'`	Idioma da letra cadastrada.

Fonte: Elaborado pelos autores (2026).

Nota: PK: chave primária; FK: chave estrangeira; AI: `AUTO_INCREMENT`.

As decisões de projeto refletidas no modelo são as seguintes:

- **Normalização:** cada atributo não chave depende apenas da chave da sua tabela.

Por isso, a tabela `fila` não repete o identificador nem o título do vídeo, que pertencem à música e são obtidos pela chave estrangeira `id_musica`. Pelo mesmo motivo, a URL do vídeo não é armazenada: ela é montada a partir de `youtube_id` (`https://www.youtube.com/watch?v=` seguido do identificador);

- **Registro das músicas pedidas:** quando o cliente escolhe um vídeo na pesquisa, a API procura em `musica` um registro com o mesmo `youtube_id`. Se ele não existir, a música é cadastrada automaticamente com o identificador e o título retornados pelo YouTube; em seguida, o pedido é inserido em `fila` referenciando esse registro. Assim, toda música pedida passa a fazer parte do catálogo local, o que também alimenta a consulta prévia ao catálogo descrita na Seção 4.2;

- **Ordem da fila (FIFO):** a ordem de chamada é a ordem de chegada dos pedidos, dada por `dt_pedido` e, em caso de empate, por `id_fila`. A posição de cada pedido não é armazenada: ela é calculada na consulta, entre os pedidos com status `na_fila`. Um índice sobre (`status`, `dt_pedido`) atende a essa consulta. O administrador não reordena a fila; quando um cantor está ausente, o pedido é removido (status `cancelado`);

- **Regras verificadas pela API:** a RN01 (música repetida) e a RN02 (limite de pedidos) dependem de outros registros da tabela e, por isso, não são expressas como restrições do banco. Antes de inserir um pedido, a API verifica, na mesma transação, se já existe um pedido da mesma música com status `na_fila` ou `reproduzindo` e se a quantidade de pedidos ativos do cliente (status `na_fila` ou `reproduzindo`) está abaixo do limite da RN02. Esse limite é controlado pela própria API no momento da validação do pedido de música e, por isso, não é armazenado no banco de dados;

- **Estados do pedido:** os valores de `status` correspondem aos estados da RN04 (*Na Fila*, *Reproduzindo*, *Concluído* e *Cancelado*). Os estados *Submetido* e *Rejeitado* do Diagrama de Estados (Figura 9) não são persistidos, porque um pedido rejeitado na validação nunca chega a ser inserido na tabela;

- **Letras das músicas:** a tabela `letra` é mantida no modelo, mas não faz parte do escopo do MVP. Ela não é preenchida nem consultada nesta etapa e será utilizada quando a funcionalidade de letras for desenvolvida (Seção 6.3).

4.4 PROJETO DA INTERFACE DO USUÁRIO

O projeto visual das interfaces priorizou a simplicidade de uso e o tema escuro de alto contraste (RNF01), adequado ao ambiente noturno do Apache Gru Rock Bar. As telas seguem a separação de perfis definida na RN03: o cliente faz pedidos e acompanha a fila, e apenas o administrador controla a reprodução. O sistema tem três interfaces:

1. **Cliente Web:** aplicação web responsiva, acessada pelo navegador do celular a partir do QR Code exibido no telão, sem instalação (RNF03). Reúne:

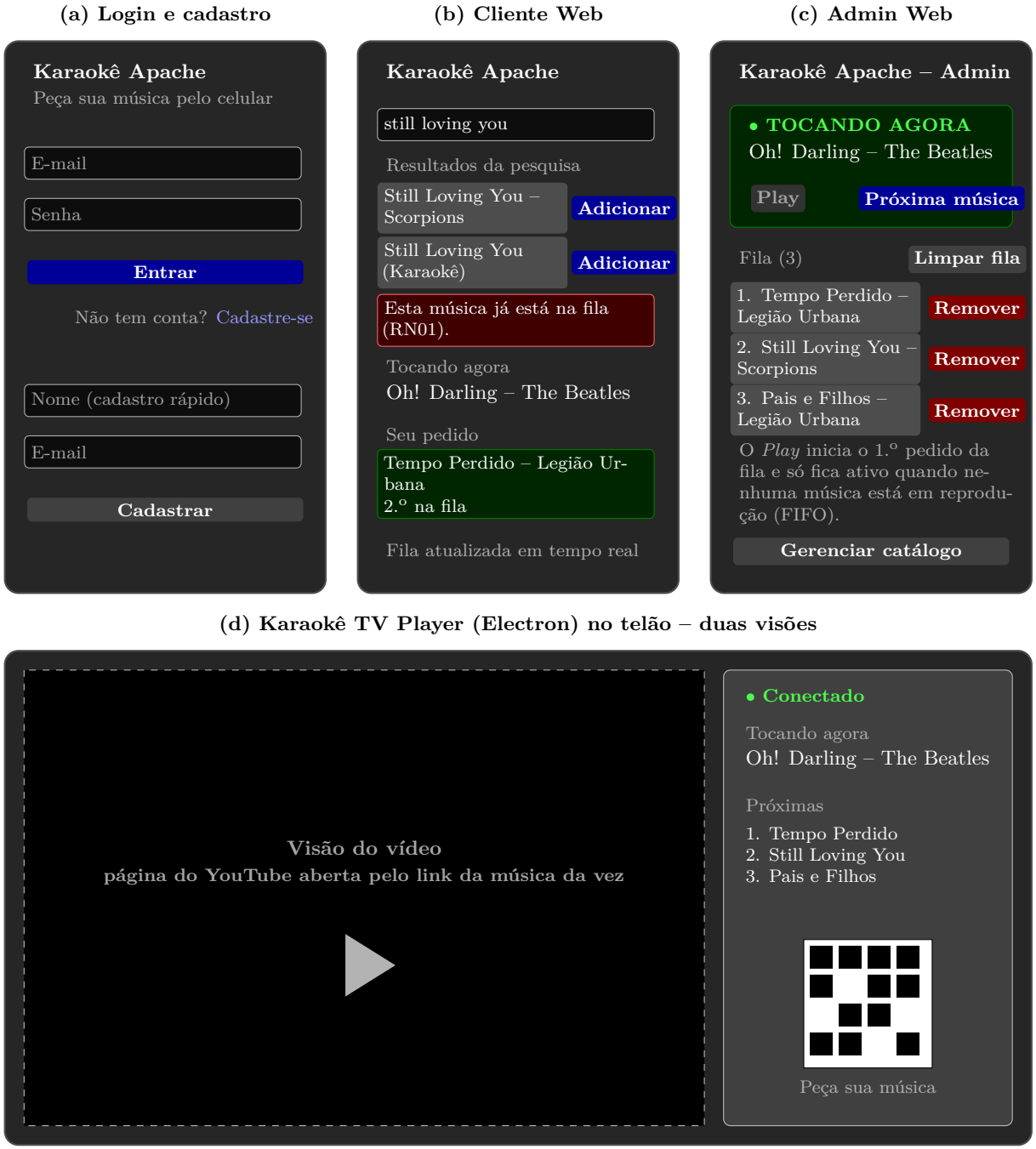
    - *Login e cadastro:* acesso com e-mail e senha ou cadastro rápido com nome e e-mail (RF01);
    - *Pesquisa e pedido:* campo de pesquisa e lista de resultados obtidos pela API, cada um com o botão **Adicionar** (RF02 a RF04);
    - *Acompanhamento:* música em reprodução e posição do pedido do cliente na fila (RF06);
    - *Alertas de validação:* mensagens exibidas quando um pedido é recusado, como música já presente na fila (RN01) ou limite de pedidos atingido (RN02), conforme o RF12.

2. **Admin Web:** aplicação web acessada pelo organizador após login com perfil `admin`. Apresenta o *card Tocando agora*, com os botões **Play** e **Próxima música**, e a fila numerada em ordem de chegada, com o botão **Remover** em cada pedido e o botão **Limpar fila**. Como a fila segue a ordem de chegada (FIFO), o *Play* atua apenas sobre o primeiro pedido e só fica disponível quando nenhuma música está em reprodução. A gestão do catálogo (RF11) fica em uma área própria dessa interface;

3. **Karaokê TV Player (Electron):** janela exibida no telão, dividida em duas visões. A visão do vídeo, que ocupa a maior parte da tela, abre a página do YouTube pelo link da música da vez. A visão da interface própria do sistema mostra o indicador de conexão (RF12), a música em reprodução, as próximas músicas da fila e o QR Code de acesso ao Cliente Web. O identificador técnico do vídeo não é exibido aos usuários.

A Figura 6 apresenta o *wireframe* dessas interfaces.

Figura 6 – *Wireframe* das Interfaces do Sistema



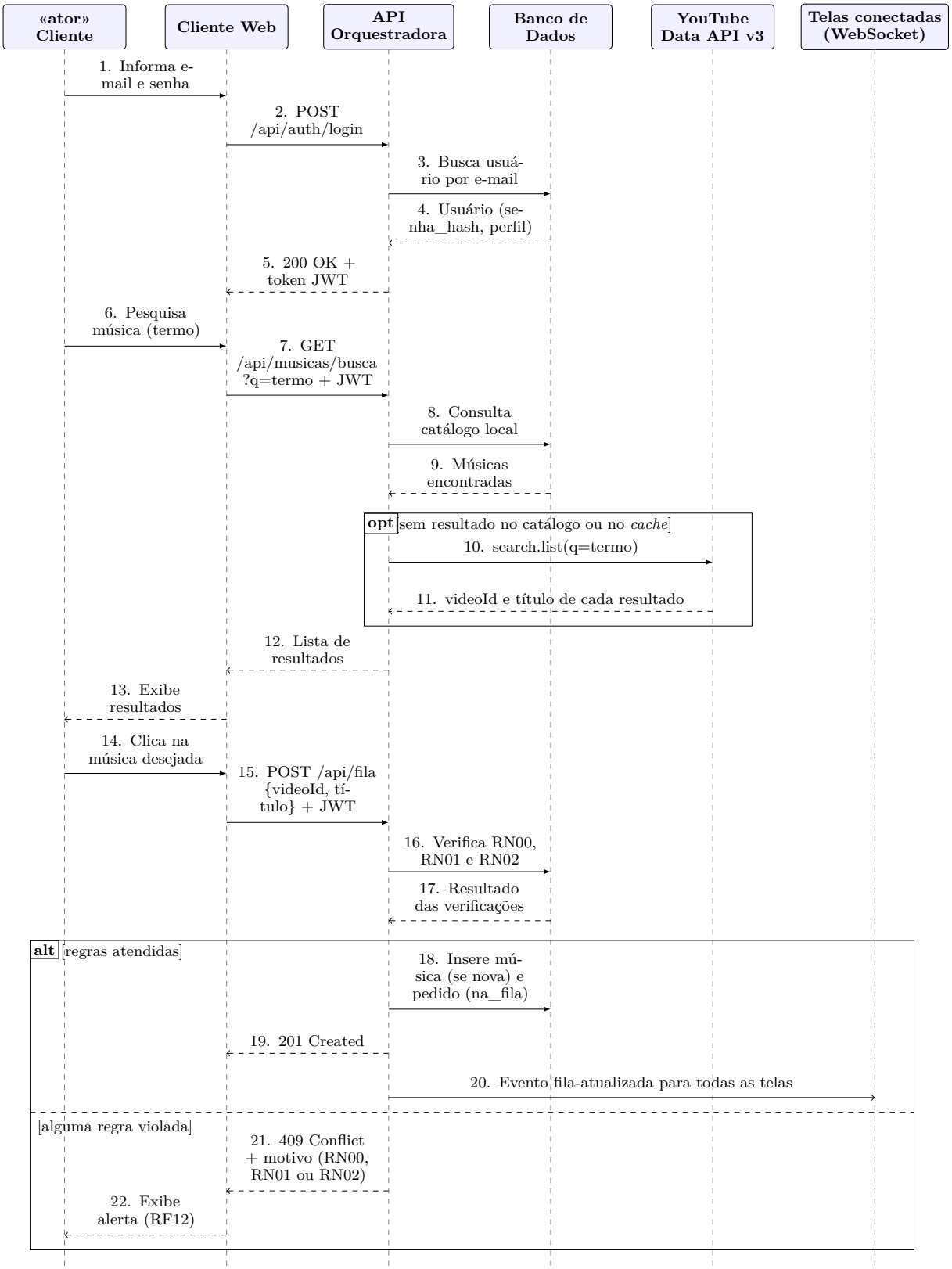
Fonte: Elaborado pelos autores (2026).

4.5 DIAGRAMAS DE INTERAÇÃO

Os Diagramas de Sequência UML a seguir mostram como os subsistemas colaboram ao longo do tempo. O cenário foi dividido em dois diagramas, um para cada ator: o pedido de música feito pelo cliente (Figura 7) e o controle do palco feito pelo administrador, com a consequente reprodução no telão (Figura 8).

Em ambos, a comunicação segue duas regras. Os comandos (login, pesquisa, pedido e controle da fila) são requisições HTTP à API Orquestradora, sob o prefixo `/api`, e as requisições de usuários autenticados levam o token JWT obtido no login. As mudanças na fila são comunicadas pela API, por meio de **WebSocket** (FETTE; MELNIKOV, 2011) com o protocolo de mensagens **STOMP** (STOMP, 2012), com o evento `fila-atualizada`, enviado a **todas as telas conectadas** (Cliente Web, Admin Web e Electron), e não apenas a quem fez a requisição. Esse envio imediato é o que permite atender ao RNF02 (atualização em menos de 2 segundos).

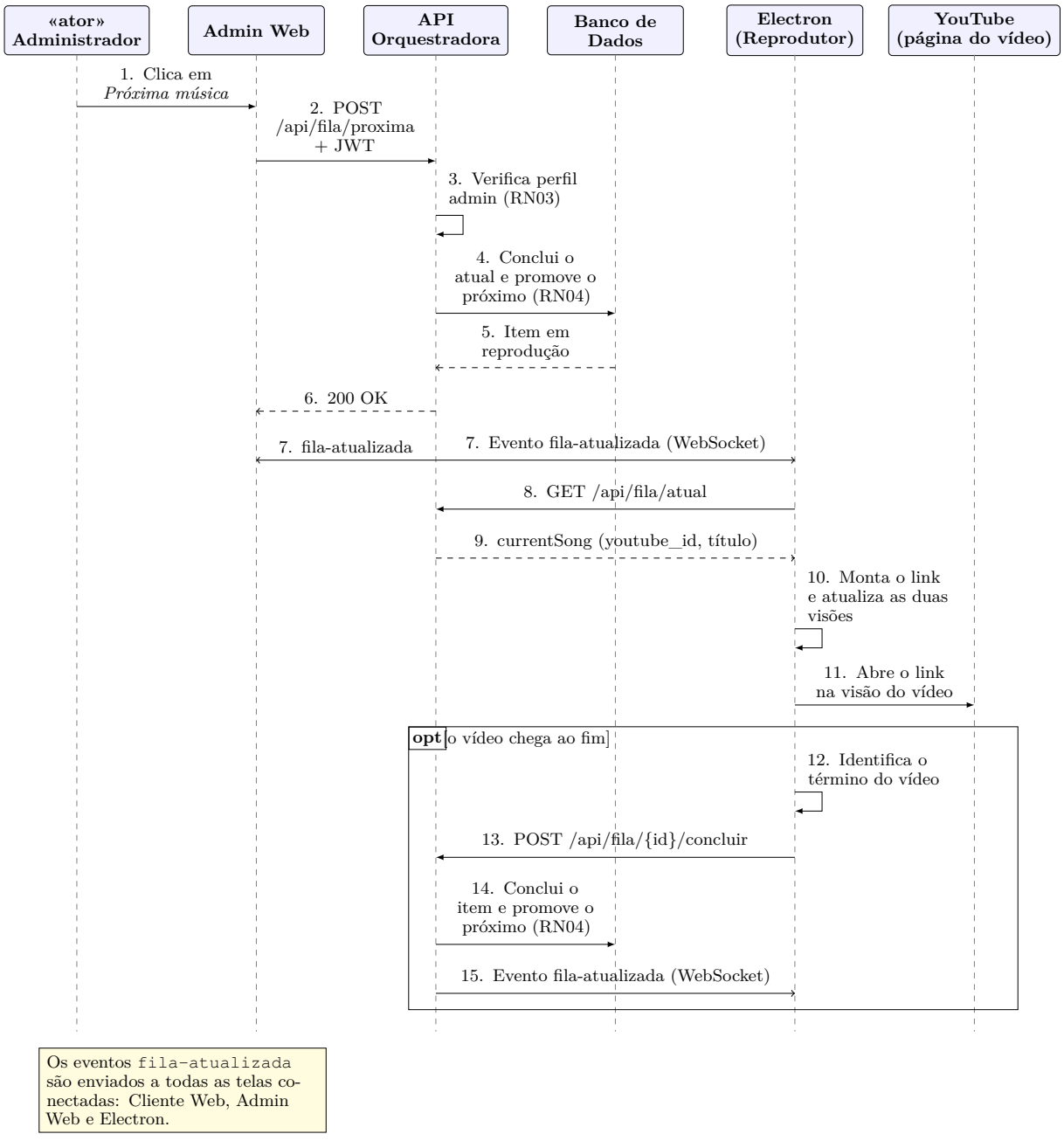
Figura 7 – Diagrama de Sequência UML – Pedido de Música pelo Cliente



Fonte: Elaborado pelos autores (2026).

No pedido, o `videoId` vem da pesquisa no YouTube (mensagens 10 e 11), e não da API do sistema. A consulta ao YouTube só ocorre quando a música não é encontrada no catálogo local nem no *cache* (fragmento *opt*), o que economiza a cota diária da *YouTube Data API v3*. Na mensagem 16, a API verifica separadamente a disponibilidade da música (RN00), a existência de um pedido ativo da mesma música (RN01) e o limite de pedidos ativos do cliente (RN02). Se alguma dessas regras for violada, o pedido não é inserido e o cliente recebe um alerta com o motivo (fragmento *alt*).

Figura 8 – Diagrama de Sequência UML – Controle do Palco e Reprodução no Telão



Fonte: Elaborado pelos autores (2026).

Como a fila segue a ordem de chegada (FIFO), os comandos do administrador atuam sempre sobre o início da fila: *Play* inicia o primeiro pedido com status `na_fila` quando nenhuma música está em reprodução, e *Próxima música* conclui o pedido atual e promove o seguinte. Ao receber o evento `fila-atualizada`, o Electron busca na API a faixa ativa (mensagens 8 e 9) e abre o vídeo pelo link na visão correspondente. O fragmento *opt* representa o aviso de término do vídeo enviado pelo Reprodutor por meio de `IEventoReproducao`, que conclui o pedido e promove o próximo; no MVP atual, esse avanço automático ainda é um incremento previsto, e o avanço é feito pelo administrador.

4.6 TECNOLOGIAS UTILIZADAS

A seleção tecnológica do projeto priorizou ferramentas modernas, multiplataforma e amplamente adotadas pelo mercado de desenvolvimento de software, consolidadas na Tabela 10.

Tabela 10 – Tecnologias, frameworks e ferramentas utilizadas no projeto.

Categoria	**Tecnologia / Ferramenta**	**Função Desempenhada no Projeto**
Front-end Web	React 19 + Vite (REACT, 2025)	Construção reativa da interface de controle e fila (*Single Page Application*), com atualização instantânea de estado e empacotamento rápido via Vite.
Aplicação Desktop (Telão)	Electron (ELECTRON, 2025)	Encapsulamento nativo do reprodutor *Karaokê TV Player*, garantindo execução dedicada em tela cheia, controle de políticas de rede e isolamento de contexto.
Back-end / API Orquestradora	Java 21 + Spring Boot (SPRING, 2026)	Desenvolvimento da API RESTful centralizada, aplicação das regras de negócio (RN00–RN04), autenticação e orquestração da fila de reprodução. O Spring Boot fornece a base da aplicação: exposição dos *endpoints* HTTP, acesso ao banco de dados MySQL e canal WebSocket para o envio dos eventos da fila.
Hospedagem	Provedor de nuvem (a definir)	Execução da aplicação web, da API Orquestradora e do banco de dados MySQL; o Electron é executado localmente no computador do estabelecimento (Seção 5.4).
Comunicação em Tempo Real	WebSocket (FETTE; MELNIKOV, 2011) com STOMP (STOMP, 2012)	Envio do evento `fila-atualizada` da API para todas as telas conectadas (Cliente Web, Admin Web e Electron) sempre que a fila muda, atendendo ao RNF02.
Banco de Dados	MySQL	Sistema de Gerenciamento de Banco de Dados relacional utilizado para a persistência estruturada das tabelas `usuario`, `musica` e `fila`, conforme o modelo de dados da Seção 4.3.
Modelagem de Engenharia	brModelo, UMLet (`.uxf`) e Draw.io	Construção do Modelo Entidade-Relacionamento conceitual e dos diagramas arquiteturais e UML do sistema.
Versionamento de Código	Git e GitHub	Controle de versão distribuído e colaboração entre os integrantes da equipe no repositório do projeto (`MurilloAssis/Projeto-Extensao-Karaoke`).
Gestão e Documentação	Trello e LaTeX / Overleaf	Acompanhamento das tarefas ágeis em quadro Kanban e editoração científica do relatório segundo as normas da ABNT.

Fonte: Elaborado pelos autores (2026).

4.7 SEGURANÇA E CONTROLE DE ACESSO

O projeto de segurança e controle de acesso do sistema contempla quatro camadas de proteção, alinhadas às boas práticas de desenvolvimento seguro e à Lei Geral de Proteção de Dados Pessoais – LGPD (Lei nº 13.709/2018) (BRASIL, 2018a):

1. **Autenticação e Controle de Acesso Baseado em Papéis (RBAC):** distinção entre sessões de `cliente` (autorizadas apenas a consultar o catálogo/fila e submeter pedidos sob as regras RN00–RN02) e sessões de `admin` (autorizadas a controlar

a reprodução do palco conforme a RN03), utilizando senhas com hash `bcrypt` e tokens JWT;

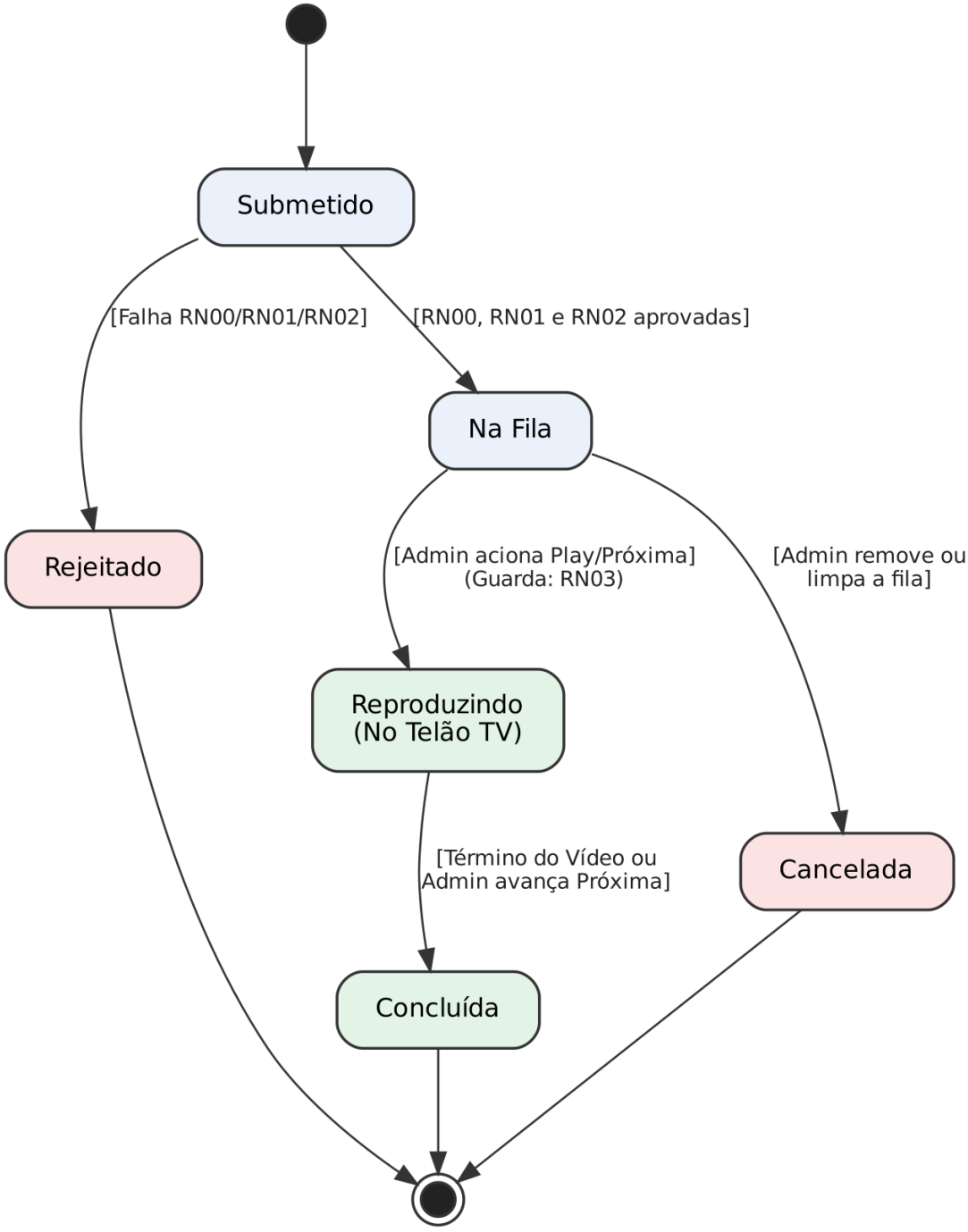
2. **Segurança no Processo Electron (*Context Isolation* e CSP):** as duas visões da janela do *Karaokê TV Player* são configuradas com `contextIsolation: true`, `nodeIntegration: false` e `sandbox: true`. A visão da interface própria do sistema aplica uma *Content Security Policy* (CSP) que restringe a origem de scripts à aplicação local (`script-src`) e as conexões de rede à API Orquestradora (`connect-src`). A visão do vídeo carrega uma página externa do YouTube e, por isso, não recebe nenhum acesso privilegiado do Electron; sua navegação é limitada ao domínio do YouTube, e tentativas de abrir outros endereços ou novas janelas são bloqueadas;

3. **Isolamento de Variáveis de Ambiente e Credenciais de Banco:** armazenamento das credenciais de conexão ao MySQL e chaves de assinatura em variáveis de ambiente isoladas do código-fonte via `.gitignore`;

4. **Minimização de Dados (Art. 6º da LGPD):** coleta restrita ao nome de chamada ao palco e e-mail de identificação do participante.

A Figura 9 apresenta o **Diagrama de Estados UML** que formaliza as transições de estado permitidas para um pedido musical dentro da fila (RN04) e as guardas de autorização (RN03) exigidas em cada transição.

Figura 9 – Diagrama de Estados UML – Ciclo de Vida de um Pedido Musical na Fila (RN03 e RN04)



Fonte: Elaborado pelos autores (2026).

5 IMPLEMENTAÇÃO

Este capítulo descreve a materialização do projeto **Karaokê Apache** em software. O projeto encontra-se em **fase de Produto Mínimo Viável (MVP)**, e a implementação ocorreu em duas etapas: uma **Prova de Conceito (PoC)**, que validou a integração entre a interface web e o reprodutor Electron, e o **MVP**, em desenvolvimento, que implementa a arquitetura definida no Capítulo 4. As seções a seguir distinguem o que foi validado na PoC do que está em implementação no MVP e apresentam o ambiente utilizado, a organização da aplicação, a integração entre os módulos, os testes, os resultados e as limitações atuais.

5.1 AMBIENTE DE DESENVOLVIMENTO

O desenvolvimento e a execução da PoC e do MVP foram conduzidos em um ambiente padronizado entre os integrantes da equipe, composto pelos seguintes recursos:

- **Estações de Trabalho e Sistemas Operacionais:** computadores pessoais dos desenvolvedores e máquinas dos laboratórios de informática do IFSP – Campus São Paulo, executando sistemas operacionais Windows 10/11 (64 bits) e distribuições Linux;
- **Linguagens, Banco de Dados e Ferramentas de Construção:** kit de desenvolvimento *Java 21 (JDK 21)* e framework *Spring Boot* para a API de back-end, Sistema de Gerenciamento de Banco de Dados *MySQL*, ambiente *Node.js* (com gerenciador de pacotes `npm`), empacotador *Vite* e framework *Electron* para o front-end e o reprodutor do telão;
- **IDEs e Repositório de Código no GitHub:** desenvolvimento utilizando *Visual Studio Code* / IDEs Java e versionamento distribuído centralizado no repositório oficial <https://github.com/MurilloAssis/Projeto-Extensao-Karaoke>.

5.2 ESTRUTURA DA APLICAÇÃO

A aplicação está organizada em três partes, que correspondem aos subsistemas definidos na arquitetura (Seção 4.2):

- **Aplicação web (React e Vite):** reúne as interfaces do Cliente Web e do Admin Web, com as telas de login, pesquisa e pedido de músicas, acompanhamento da fila e controle do palco. Comunica-se com a API por HTTP e recebe as atualizações da fila por WebSocket;

- **API Orquestradora (Java 21 e Spring Boot):** concentra as regras de negócio (RN00 a RN04), a autenticação, o acesso ao banco de dados MySQL, a integração com a *YouTube Data API v3* e o canal WebSocket por onde são enviados os eventos da fila;
- **Reprodutor (Electron):** contém o processo principal do aplicativo desktop *Karaokê TV Player*, responsável pela janela exibida no telão, e a interface própria do reprodutor.

5.3 IMPLEMENTAÇÃO DOS COMPONENTES

A implementação priorizou os componentes necessários para validar a proposta central do sistema: receber pedidos musicais, organizar a fila sem duplicidades e controlar remotamente o que é reproduzido na TV do bar.

- **Componente *Pedido de Música*:** na PoC, o vídeo escolhido era informado pela interface web, que extraía o identificador do vídeo e bloqueava a inclusão quando o vídeo era inválido ou indisponível (RN00) ou quando o identificador já constava na fila ativa ou em reprodução (RN01). No MVP, a pesquisa passa a ser feita pela API, por meio da *YouTube Data API v3*, e as validações, incluindo o limite de pedidos por cliente (RN02), passam a ser executadas pela API (Seções 4.2 e 4.3);
- **Componente *Fila*:** na PoC, foi implementada a gestão da lista de músicas (`queue`) e da faixa ativa (`currentSong`). A interface permite adicionar itens ao final da fila, acionar os botões **Play** e **Próxima música**, excluir itens individuais e limpar toda a fila. As alterações eram propagadas às telas conectadas por **WebSocket com o protocolo STOMP** (STOMP, 2012). No MVP, a fila passa a ser persistida no banco de dados, em ordem de chegada (FIFO);
- **Componente *Reprodutor – Karaokê TV Player*:** foi implementado o processo principal do Electron (`main.js`), que instancia uma janela dedicada (`BrowserWindow`) responsável pela reprodução. O *Karaokê TV Player* recebe pelo WebSocket as mudanças da faixa ativa e, a cada nova música, exibe o status de conexão e carrega o vídeo correspondente. Na PoC, o vídeo era incorporado à página do reprodutor; as falhas de reprodução causadas por restrições de direitos autorais levaram à decisão de abrir o vídeo pelo link, em uma janela dividida em duas visões (Seção 4.1), que está em implementação no MVP;
- **Componente *Usuários* e persistência relacional (Java 21, Spring Boot e MySQL):** está em implementação no MVP a API em Java 21 com Spring Boot, integrada ao banco de dados MySQL, para o cadastro e a autenticação de

usuários (`admin`/`cliente`) e a persistência das tabelas `usuario`, `musica` e `fila`, conforme o modelo de dados da Seção 4.3.

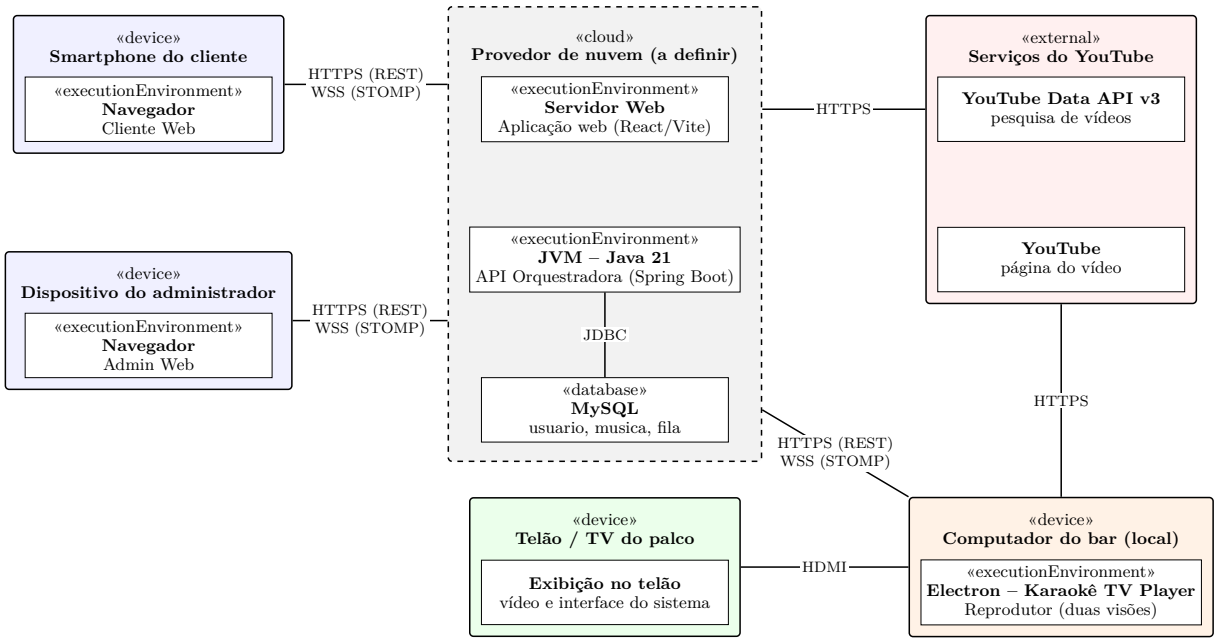
5.4 INTEGRAÇÃO DOS MÓDULOS

No MVP, a aplicação web e a API Orquestradora, com o banco de dados MySQL, serão executadas em um **provedor de nuvem**, ainda não definido pela equipe. O **Karaokê TV Player** em Electron é executado localmente, no computador do estabelecimento conectado ao telão do palco. Os clientes e o administrador acessam as interfaces web pelo navegador de seus dispositivos, e todos os módulos comunicam-se com a API pela internet. A integração segue dois canais:

- **Comandos por HTTP:** o cadastro de usuários, a pesquisa, os pedidos de música e o gerenciamento da fila são requisições das interfaces web à API Orquestradora;
- **Atualizações por WebSocket (STOMP):** sempre que a fila muda, a API publica o evento `fila-atualizada`, recebido por todas as telas inscritas, incluindo o Electron, que então busca a faixa ativa e reproduz o vídeo. Esse mecanismo, já utilizado na PoC, é o que permite que uma adição ou troca de música feita na interface web apareça imediatamente no telão.

A Figura 10 apresenta o **Diagrama de Implantação UML** com a topologia de execução do MVP. Na PoC, todos os módulos eram executados localmente, no mesmo computador.

Figura 10 – Diagrama de Implantação UML – Topologia de Execução do MVP



Fonte: Elaborado pelos autores (2026).

5.5 TESTES REALIZADOS

Os testes desta etapa são testes de sistema manuais, conduzidos pela equipe sobre os fluxos essenciais, com o objetivo de verificar se o comportamento da aplicação corresponde aos requisitos especificados (SOMMERVILLE, 2019). Os testes da PoC foram executados com a interface web e o Electron em funcionamento simultâneo e registrados em vídeo pela equipe. A Tabela 11 sumariza os testes realizados e os testes planejados para o MVP.

Tabela 11 – Testes realizados na Prova de Conceito e testes planejados para o MVP.

ID	Aspecto Avaliado	Procedimento Executado e Resultado Observado	Situação
TST-01	Escolha e adição de vídeo à fila	Seleção de vídeos de karaokê (como *Still Loving You – Scorpions* e *Oh! Darling – The Beatles*) na interface web; os itens foram validados e listados corretamente na fila numerada.	Validado na PoC (registro em vídeo)
TST-02	Sincronização do comando *Play* com o Electron	Acionamento do botão *Play* na interface web; o *Karaokê TV Player* recebeu a atualização pelo WebSocket (STOMP), mudou o status para *Conectado – Tocando* e iniciou o vídeo.	Validado na PoC (registro em vídeo)
TST-03	Avanço de faixa (*Próxima música*) e limpeza	Acionamento do botão *Próxima música* e exclusão de itens; a fila avançou na ordem de chegada e a tela do reprodutor foi atualizada.	Validado na PoC (registro em vídeo)
TST-04	Bloqueio de música duplicada na fila (RN01)	Tentativa de inserir novamente um vídeo já presente na fila ativa; a inclusão foi bloqueada e a fila permaneceu inalterada.	Validado na PoC
TST-05	Integração da API (Java 21 e Spring Boot) com o MySQL	Testes de persistência das entidades modeladas e de autenticação por perfil (`admin`/`cliente`).	Em andamento
TST-06	Limite de pedidos por cliente (RN02)	Tentar exceder o limite de pedidos ativos de um cliente; o pedido excedente deve ser recusado com alerta.	Planejado
TST-07	Controle de acesso (RN03)	Tentar acionar *Play* ou *Próxima música* com um usuário `cliente`; a API deve recusar a requisição.	Planejado
TST-08	Pesquisa pela *YouTube Data API v3*	Pesquisar músicas presentes e ausentes do catálogo local; verificar os resultados e o uso do *cache*.	Planejado
TST-09	Tempo de atualização (RNF02)	Medir o tempo entre o comando na interface web e a atualização do telão; o resultado deve ser inferior a 2 segundos.	Planejado
TST-10	Reprodução em duas visões no Electron	Verificar a abertura do vídeo pelo link em uma visão e a interface própria do sistema na outra.	Planejado

Fonte: Elaborado pelos autores (2026).

5.6 RESULTADOS OBTIDOS

Os resultados alcançados até o momento são relacionados abaixo aos objetivos específicos (Seção 1.2.2) e às necessidades identificadas no Capítulo 2:

- **Diagnóstico, especificação e modelagem (objetivos 1 a 3):** atendidos. O levantamento junto ao parceiro resultou nas regras de negócio (RN00 a RN04), nos requisitos e na modelagem UML e de dados apresentados nos Capítulos 2 a 4, com a divisão em quatro componentes (*Usuários*, *Fila*, *Pedido de Música* e *Reprodutor*);
- **Implementação da PoC e do núcleo do MVP (objetivo 4):** parcialmente atendido. A PoC integrou a interface web em React/Vite ao reprodutor em Electron;

a API em Spring Boot com MySQL, a pesquisa pelo YouTube e a reprodução em duas visões estão em implementação;

- **Sincronização em tempo real (objetivo 5 e necessidade N02):** atendido na PoC. A comunicação por WebSocket com STOMP mostrou que o operador pode gerenciar a fila na interface web enquanto o **Karaokê TV Player** exibe as músicas em uma janela dedicada à TV, sem procurar vídeos manualmente nem arrastar abas para o telão. A medição formal do tempo de atualização (RNF02) está planejada (TST-09);

- **Prevenção de repetições (necessidade N03):** parcialmente atendido. O bloqueio de músicas duplicadas (RN01) foi validado; o limite de pedidos por cliente (RN02) será validado no MVP;

- **Autonomia na escolha de vídeos e reprodução no telão (necessidades N01 e N04):** parcialmente atendido. A escolha do vídeo pelo sistema e o controle remoto do telão foram validados na PoC; o acesso do cliente pelo celular com login, a pesquisa pelo YouTube e o avanço automático ao fim do vídeo dependem das etapas seguintes;

- **Custo (objetivo geral):** atendido. A solução tem baixo custo para o estabelecimento: as ferramentas utilizadas são gratuitas ou de código aberto, sem custo de licenciamento, e a *YouTube Data API v3* é gratuita dentro da cota diária do serviço (tratada como limitação na Seção 5.7). O único custo previsto é o da plataforma de hospedagem em nuvem da aplicação web, da API e do banco de dados, cujo valor dependerá do provedor escolhido e do plano contratado.

5.7 LIMITAÇÕES DA IMPLEMENTAÇÃO

Por tratar-se de uma versão em fase de MVP, registram-se as seguintes limitações atuais, que orientam a continuidade do desenvolvimento:

- **Persistência e autenticação:** na PoC, a fila e a faixa ativa não eram persistidas no banco de dados relacional. A integração das telas de login e cadastro com a API e o MySQL está em finalização no MVP;

- **Reprodução em duas visões:** a PoC exibia o vídeo incorporado à página do reprodutor. A abertura do vídeo pelo link, em uma das duas visões da janela do Electron, está em implementação;

- **Transição automática ao final do vídeo:** o avanço para a próxima música é acionado manualmente pelo organizador no botão *Próxima música*. O avanço

automático depende de o Reprodutor identificar o término do vídeo na visão do YouTube, mecanismo ainda a ser definido pela equipe;

- **Cota da *YouTube Data API v3*:** a cota padrão limita o sistema a cerca de 100 pesquisas por dia (GOOGLE, 2026). O catálogo local e o *cache* reduzem esse consumo, mas não eliminam o limite;
- **Dependência de internet:** a aplicação web e a API são executadas em nuvem, e os vídeos são carregados do YouTube durante a apresentação. Sem conexão ativa no estabelecimento, nem os pedidos nem a reprodução funcionam;
- **Provedor de nuvem não definido:** a escolha do provedor que hospedará a aplicação web, a API e o banco de dados ainda está pendente;
- **Mecanismos de segurança pendentes:** HTTPS, CORS e limitação de requisições (*rate limiting*) ainda não foram implementados.

6 CONCLUSÃO

Este capítulo encerra o documento com as considerações finais sobre o desenvolvimento do projeto **Karaokê Apache** até a fase atual de MVP, relacionando os resultados aos objetivos estabelecidos, relatando as principais dificuldades enfrentadas pela equipe e apresentando as possibilidades de evolução do sistema.

6.1 CONSIDERAÇÕES FINAIS

O desenvolvimento do projeto **Karaokê Apache** mostrou como conceitos de Engenharia de Software, Análise de Sistemas e Arquitetura de Aplicações Distribuídas podem ser aplicados à realidade de um pequeno negócio local. A partir da rotina da *Noite de Karaokê* no estabelecimento parceiro **Apache Gru Rock Bar**, em Guarulhos – SP, problemas operacionais concretos – antes resolvidos com anotações em papel e janelas de navegador manipuladas manualmente – foram traduzidos em requisitos, em regras de negócio (RN00 a RN04) e em uma arquitetura dividida em quatro subsistemas.

Em relação ao objetivo geral, o projeto entregou a especificação completa do sistema e uma Prova de Conceito funcional, mas o MVP ainda não está concluído. Os objetivos específicos de diagnóstico, especificação e modelagem foram atendidos. A Prova de Conceito validou a sincronização em tempo real entre a fila web em React/Vite e o reprodutor desktop **Karaokê TV Player** em Electron, por meio de WebSocket com o protocolo STOMP. Permanecem em implementação a API em Spring Boot com persistência em MySQL, a pesquisa de músicas pela *YouTube Data API v3* e a reprodução em duas visões no Electron, conforme detalhado nas Seções 5.7 e 6.3. Ao longo do trabalho, o projeto também contribuiu para a formação prática dos estudantes do IFSP, que conduziram todas as etapas, do levantamento junto ao parceiro à implementação.

6.2 DIFICULDADES ENCONTRADAS

Ao longo do ciclo de levantamento, modelagem e implementação do MVP, a equipe enfrentou quatro dificuldades principais:

1. **Sincronização em tempo real entre a fila web e o reprodutor desktop:** as ações de controle da fila precisavam aparecer imediatamente na janela do *Karaokê TV Player*, sem recarregar a página. A solução adotada foi a comunicação por WebSocket com o protocolo STOMP: a cada mudança, o evento `fila-atualizada` é enviado a todas as telas, e o Electron busca a faixa ativa para reproduzi-la;

2. **Falhas na reprodução de vídeos incorporados:** muitos vídeos de karaokê apresentavam falhas quando incorporados a uma página, em razão de restrições de

direitos autorais definidas pelos detentores do conteúdo. A equipe passou então a abrir o vídeo diretamente pelo seu link, em uma das duas visões da janela do Electron, deixando a outra para a interface própria do sistema;

3. **Limite de uso da API de pesquisa do YouTube:** a cota diária da *YouTube Data API v3* permite cerca de 100 pesquisas, número baixo para uma noite de karaokê. Isso levou à decisão de consultar primeiro o catálogo local e manter um *cache* das pesquisas recentes, recorrendo ao YouTube apenas quando necessário;

4. **Calibragem do escopo para o calendário semestral:** foi necessário delimitar o escopo do MVP, descartando ideias iniciais que criariam barreiras de uso para os clientes do bar (como obrigar o download de um aplicativo nativo nas lojas móveis) em favor de uma interface web acessível via QR Code exibido no telão. Pelo mesmo motivo, a exibição de letras foi mantida no modelo de dados, mas deixada fora do MVP.

6.3 TRABALHOS FUTUROS

Após a conclusão e a implantação do MVP no Apache Gru Rock Bar, destacam-se os seguintes trabalhos futuros:

- **Avanço automático ao término do vídeo:** definir o mecanismo pelo qual o Reprodutor identifica o fim do vídeo na visão do YouTube e enviar esse aviso à API, que conclui o pedido e promove o próximo da fila, dispensando o acionamento manual do botão *Próxima música*;

- **Escolha do provedor de nuvem:** selecionar o provedor que hospedará a aplicação web, a API e o banco de dados, considerando custo, desempenho e facilidade de manutenção, e realizar a implantação do MVP nesse ambiente;

- **Módulo de letras no celular e painel de estatísticas musicais:** utilizar a tabela `letra`, já prevista no modelo de dados mas fora do escopo do MVP, para exibir as letras no celular do cliente, e gerar relatórios das músicas mais pedidas a partir dos registros da tabela `fila` (`id_musica` e `dt_pedido`);

- **Notificação de proximidade na fila:** aproveitar o canal WebSocket já existente para emitir um alerta no celular do cliente quando faltar apenas uma música para sua chamada ao palco;

- **Interação da plateia e integração com o cardápio digital da casa:** permitir que as mesas enviem reações visuais para o telão durante as apresentações e disponibilizar um atalho na tela da fila para o cardápio de bebidas e porções do bar.

REFERÊNCIAS

BALDO, R. **Micro e pequenas empresas foram responsáveis por sete em cada 10 empregos formais gerados até novembro de 2025**. Brasília, DF: Agência Sebrae de Notícias, 2026. Publicado em 7 jan. 2026. Disponível em: <https://agenciasebrae.com.br/dados/micro-e-pequenas-empresas-foram-responsaveis-por-sete-em-cada-10-empregos-formais-gerados-ate-no>. Acesso em: 1 nov. 2026. Citado na página 18.

BRASIL. **Lei nº 11.892, de 29 de dezembro de 2008. Institui a Rede Federal de Educação Profissional, Científica e Tecnológica, cria os Institutos Federais de Educação, Ciência e Tecnologia, e dá outras providências**. Brasília, DF: Presidência da República, 2008. Disponível em: <https://www.planalto.gov.br/ccivil_03/_ato2007-2010/2008/lei/l11892.htm>. Acesso em: 1 nov. 2026. Citado na página 18.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD)**. Brasília, DF: Presidência da República, 2018. Disponível em: <https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm>. Acesso em: 12 out. 2026. Citado 3 vezes nas páginas 33, 47 e 61.

BRASIL. **Resolução CNE/CES nº 7, de 18 de dezembro de 2018. Estabelece as Diretrizes para a Extensão na Educação Superior Brasileira**. Brasília, DF: Ministério da Educação, Conselho Nacional de Educação, 2018. Diário Oficial da União, seção 1, p. 49–50, 19 dez. 2018. Citado na página 18.

CNPJ API. **CNPJ 64.687.141/0001-07: APACHE GRU ROCK BAR LTDA**. 2026. Dados públicos do Cadastro Nacional da Pessoa Jurídica. Disponível em: <https://cnpj-api.com/en/cnpj/64687141000107>. Acesso em: 1 nov. 2026. Citado na página 22.

ELECTRON. **Electron Documentation: Process Model and Context Isolation**. 2025. Disponível em: <https://www.electronjs.org/docs>. Acesso em: 15 out. 2026. Citado na página 47.

FETTE, I.; MELNIKOV, A. **RFC 6455: The WebSocket Protocol**. Internet Engineering Task Force (IETF), 2011. Disponível em: <https://www.rfc-editor.org/rfc/rfc6455>. Acesso em: 1 nov. 2026. Citado 2 vezes nas páginas 43 e 47.

FOLHA DE PERNAMBUCO. **Pesquisa mostra que 39% das MPMEs ainda controlam despesas de forma manual**. Recife: Folha de Pernambuco, 2025. Publicado em 20 jan. 2025. Disponível em: <https://www.folhape.com.br/economia/pesquisa-mostra-que-39-das-mpmes-ainda-controlam-despesas-de-forma/385706/>. Acesso em: 1 nov. 2026. Citado 2 vezes nas páginas 18 e 21.

FOWLER, M. **Padrões de Arquitetura de Aplicações Corporativas**. Porto Alegre: Bookman, 2006. Citado na página 34.

GOOGLE. **YouTube Data API v3: Quota and Compliance Audits**. 2026. Disponível em: <https://developers.google.com/youtube/v3/guides/quota_and_compliance_audits>. Acesso em: 1 nov. 2026. Citado 4 vezes nas páginas 32, 34, 37 e 56.

JORNAL DO COMÉRCIO. **Bares e restaurantes estão mais digitais na gestão, mostra pesquisa**. Porto Alegre: Jornal do Comércio, 2023. Publicado em 31 maio 2023. Disponível em: <https://www.jornaldocomercio.com/economia/2023/05/1108971-bares-e-restaurantes-estao-mais-digitais-na-gestao-mostra-pesquisa.html>. Acesso em: 1 nov. 2026. Citado 2 vezes nas páginas 18 e 21.

JUKS. **Juks: Jukebox Digital para Bares e Restaurantes**. 2026. Disponível em: <https://juks.com.br/>. Acesso em: 1 nov. 2026. Citado 2 vezes nas páginas 21 e 22.

KARAFUN BUSINESS. **KaraFun Business: Karaoke Solutions for Professionals**. Lille: Recisio, 2026. Disponível em: <https://business.karafun.com/>. Acesso em: 1 nov. 2026. Citado na página 21.

KARAFUN BUSINESS. **KaraFun Stage: Pricing**. Lille: Recisio, 2026. Disponível em: <https://business.karafun.com/stage/pricing>. Acesso em: 1 nov. 2026. Citado 2 vezes nas páginas 21 e 22.

MOTOFEST BRASIL. **Noite do Karaokê no Apache GRU Rock Bar em Guarulhos - SP**. 2026. Disponível em: <https://motofestbrasil.com.br/motofest/noite-do-karaoke-no-apache-gru-rock-bar-em-guarulhos-sp>. Acesso em: 1 nov. 2026. Citado na página 22.

O'BRIEN, J. A.; MARAKAS, G. M. **Administração de Sistemas de Informação**. Porto Alegre: AMGH, 2013. Citado na página 21.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de Software: uma abordagem profissional**. 8. ed. Porto Alegre: AMGH, 2016. Citado na página 35.

REACT. **React Documentation: Describing the UI and Managing State**. 2025. Disponível em: <https://react.dev/>. Acesso em: 15 out. 2026. Citado na página 47.

SETLIST.FM. **Apache GRU, Guarulhos, Brazil: Concert Setlists**. 2026. Disponível em: <https://www.setlist.fm/venue/apache-gru-guarulhos-brazil-5bde7f84.html>. Acesso em: 1 nov. 2026. Citado na página 22.

SINGA. **Best Karaoke Software for Business**. Helsinki: Singa, 2026. Atualizado em 14 set. 2026. Disponível em: <https://singa.com/blog/best-karaoke-software-for-business/>. Acesso em: 1 nov. 2026. Citado 2 vezes nas páginas 21 e 22.

SINGA BUSINESS. **Professional Karaoke Software for Bars, Venues and KTV Rooms**. Helsinki: Singa, 2026. Disponível em: <https://singa.com/business/>. Acesso em: 14 out. 2026. Citado na página 21.

SOMMERVILLE, I. **Engenharia de Software**. 10. ed. São Paulo: Pearson Education do Brasil, 2019. Citado 2 vezes nas páginas 34 e 53.

SPRING. **Spring Boot Reference Documentation**. 2026. Disponível em: <https://docs.spring.io/spring-boot/>. Acesso em: 1 nov. 2026. Citado na página 47.

STOMP. **STOMP Protocol Specification, Version 1.2**. 2012. Disponível em: <https://stomp.github.io/stomp-specification-1.2.html>. Acesso em: 1 nov. 2026. Citado 3 vezes nas páginas 43, 47 e 51.

ANEXO A – REFERÊNCIA NORMATIVA – ART. 6º DA LEI GERAL DE PROTEÇÃO DE DADOS PESSOAIS (LEI Nº 13.709/2018)

Transcrição do Art. 6º da Lei nº 13.709, de 14 de agosto de 2018 (LGPD) (BRASIL, 2018a), utilizado como referência normativa para as restrições de privacidade e minimização de dados cadastrais no sistema **Karaokê Apache**:

> Art. 6º As atividades de tratamento de dados pessoais deverão observar a boa-fé e os seguintes princípios:
>
> I – finalidade: realização do tratamento para propósitos legítimos, específicos, explícitos e informados ao titular, sem possibilidade de tratamento posterior de forma incompatível com essas finalidades;
>
> II – adequação: compatibilidade do tratamento com as finalidades informadas ao titular, de acordo com o contexto do tratamento;
>
> III – necessidade: limitação do tratamento ao mínimo necessário para a realização de suas finalidades, com abrangência dos dados pertinentes, proporcionais e não excessivos em relação às finalidades do tratamento de dados;
>
> VII – segurança: utilização de medidas técnicas e administrativas aptas a proteger os dados pessoais de acessos não autorizados e de situações acidentais ou ilícitas de destruição, perda, alteração, comunicação ou difusão.